CSC1031

Computer Programming 4

Object Oriented Programming

Dr Mikhail Kudriavtsev

Misha

Email: Mikhail.kudriavtsev@dcu.ie

Module overview

- Object oriented programming concepts
- Java
- System design

Assessment

- 50% Exam
- 50% Continuous Assessment
  - 15% quizzes
  - ~~25% essay 70-80 pages~~
  - 15% project
  - 20% labs

Primitive data types

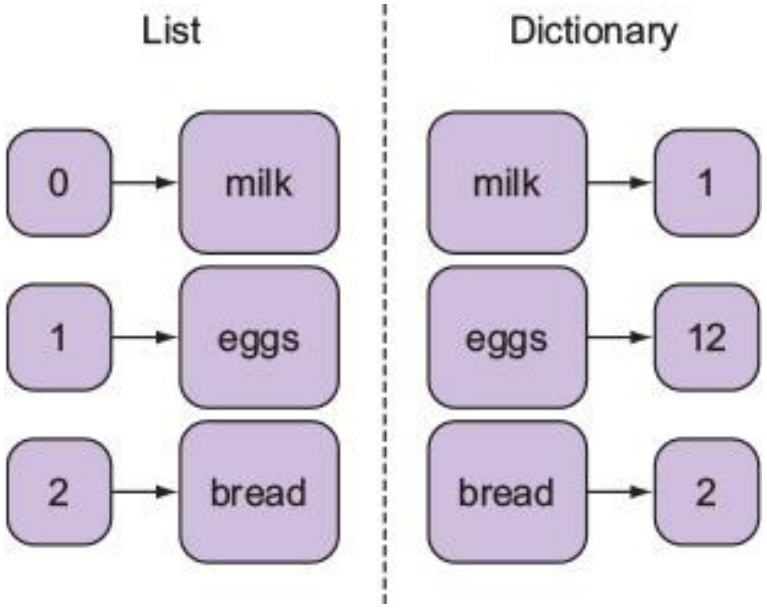
- The simplest building blocks of data in programming
- They store single, simple values
- **Examples** of Primitive Data Types
  - **Integer:** Whole numbers (e.g., 5, -100)
  - **Float:** Decimal numbers (e.g., 3.14, -0.001)
  - **String:** Sequence of characters (e.g., "Hello", "123")
- **Characteristics**
  - Fixed size in memory (language-dependent).
  - Limited functionality—only store and manipulate individual values.

From Primitive Types to Advanced Data Structures

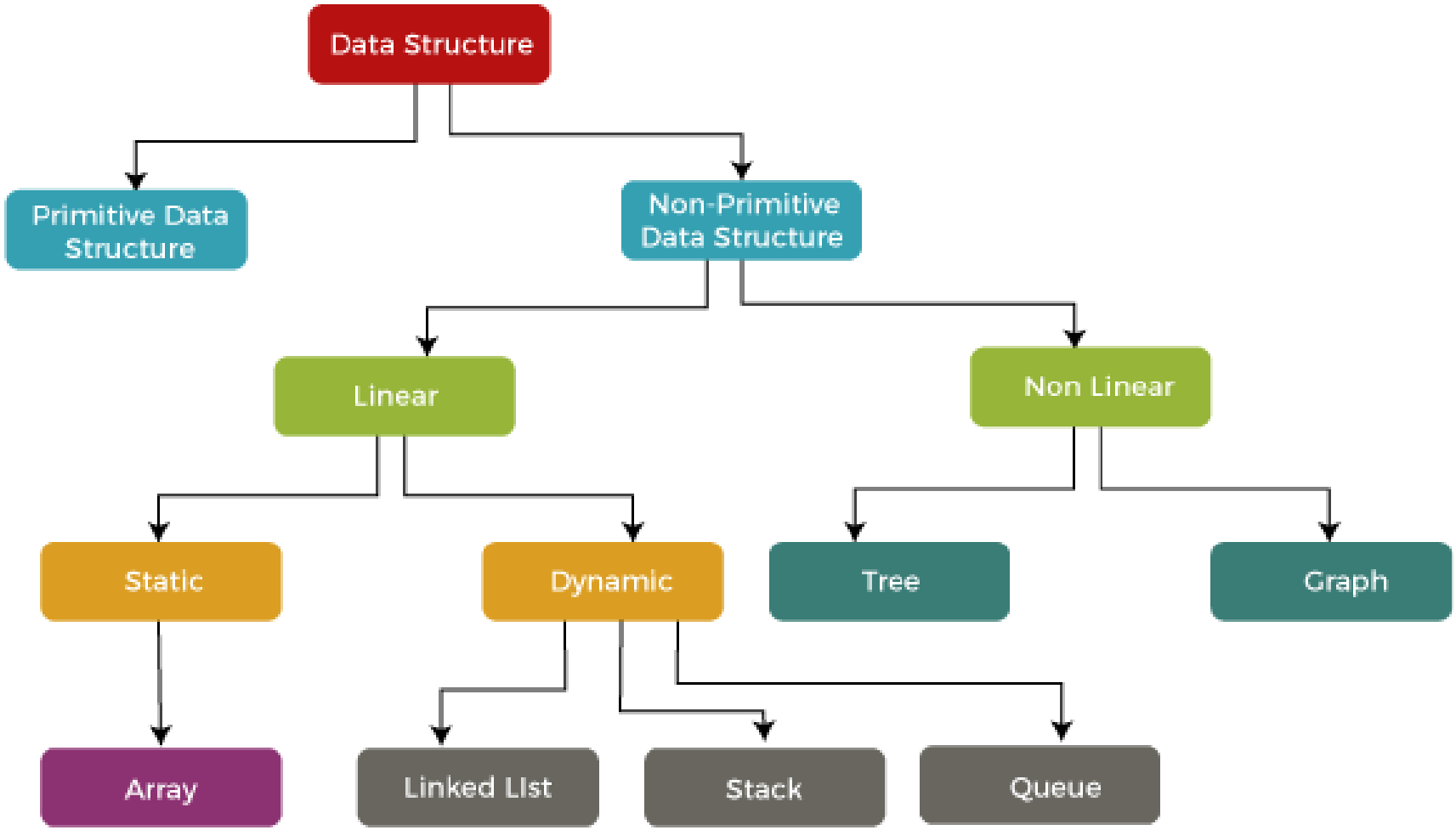
- As programs became more complex, there was a need to group related data together.
- **Arrays** were introduced to store multiple items of the same type.
  - numbers = [1, 2, 3, 4] (Python)
  - int[] numbers = {1, 2, 3, 4}; (Java)
- Arrays *typically* store data of the same type (e.g., only integers).
  - Flexible Grouping in Python: allows arrays (lists) to store mixed data types
    - a = [1, "apple", 3.14]

Dictionary - Associative array

- To represent relationships between data, key-value pairs were introduced




```python
person = {"name": "Alice", "age": 25, "city": "Dublin"}
```


Data Structure
Primitive Data Structure
Non-Primitive Data Structure
Linear
Non Linear
Static
Dynamic
Tree
Graph
Array
Linked LIst
Stack
Queue

Example: Representing a Dot in 3D Space

- **Problem:**
  - We want to represent a dot in 3D space with its x, y, and z coordinates.
- **Solution:**
  - Use a simple list to store the coordinates

```python
dot = [1.0, 2.0, 3.0]  # [x, y, z]
```

Adding More Information

- **Task:**
  - Assign a name to each dot in addition to the coordinates.
- **Solution**
  - Use a dictionary to group related data:

```python
dot = {
    "name": "DotA",
    "coordinates": [1.0, 2.0, 3.0]
}
```

Adding More Properties

- **Task:**
  - Add properties like color and whatever
- **Solution:**
  - Extend the dictionary to include more attributes:

```python
dot = {
    "name": "DotA",
    "coordinates": [1.0, 2.0, 3.0],
    "color": "red",
    "visibility": True
}
```

Nested Dictionaries

python

```
dots = [
    {
        "name":
        "properti
            "coo
            "col
            "visibility": True
        }
    },
    {
        "name": "DotB",
        "properties": {
```

python

```
# Change visibility of DotB
dots[1]["properties"]["visibility"] = True
```

- **Challenges with Nested Dictionaries**
  - Difficult to manage when the number of dots grows.
  - No type checking—easy to misspell keys or forget attributes.
  - Code becomes harder to read and maintain
- **Example of Maintenance Complexity**
  - Adding a new property (e.g., size) requires updating every dot individually.

- It is chaotic

Creating Our Own Type – A Dot

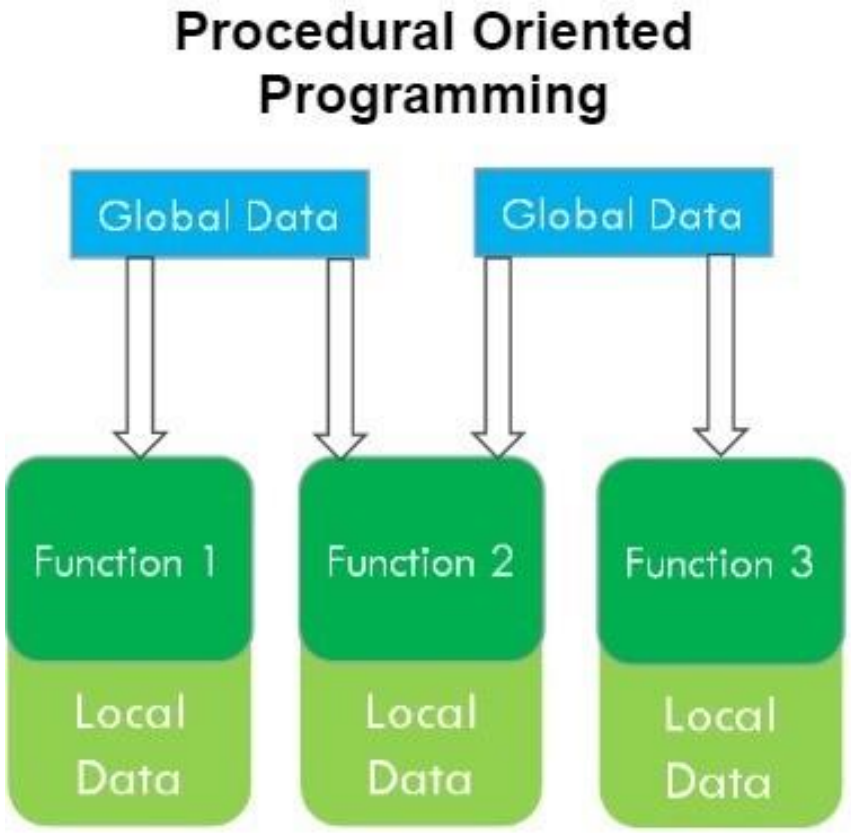
- We can optimize by creating our own type to represent a dot.
- A **structure** (or equivalent concept) groups related data under one entity.
- **What is a Structure?**
  - A structure is a **blueprint** for organizing related data together.
  - It defines **attributes** but does not include **behaviors**
- **Advantages of our own Structures**
  - Enforces consistency:
    - Every dot will have the same attributes.
  - Easier to maintain:
    - Changes in the structure affect all instances.
  - Scalable:
    - Manage large data sets without repeating code.

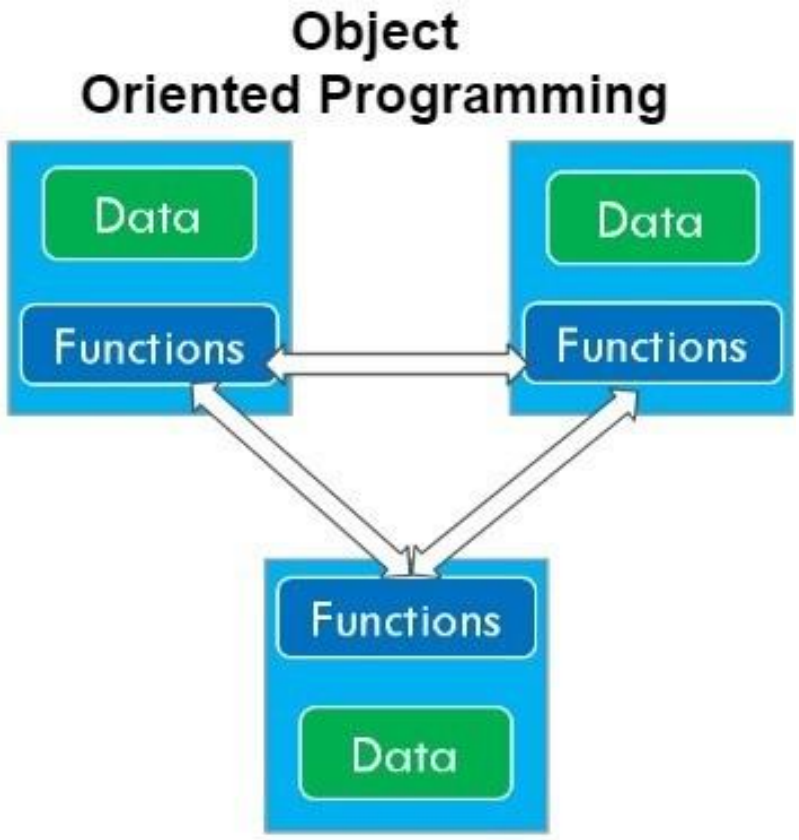
```c
struct Dot {
    char name[20];
    float x, y, z;
    char color[10];
    bool visibility;
};
```

Object Oriented Programming
OOP

What is OOP

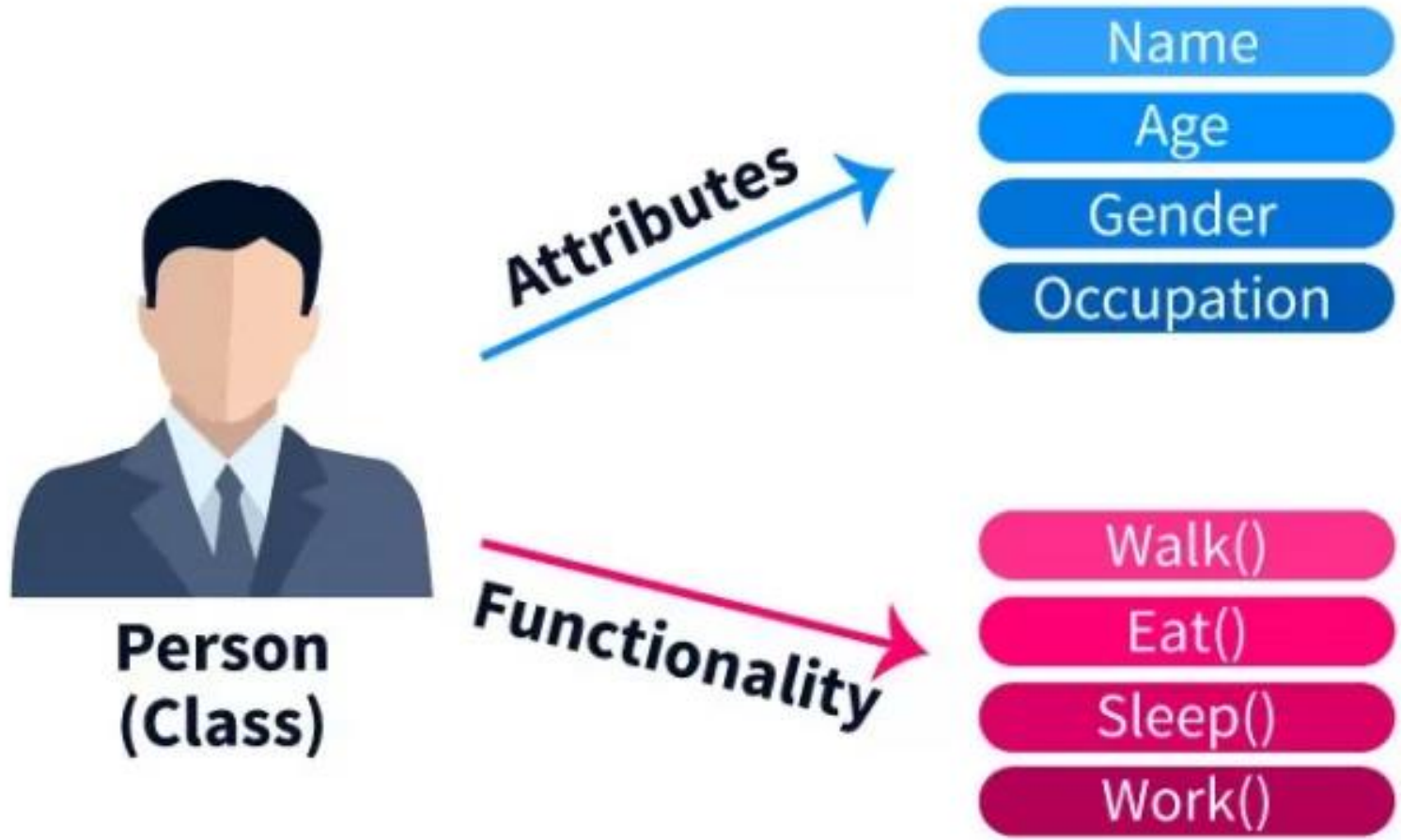
- Object-oriented programming (OOP) is a computer a programming **paradigm** that organizes software design **around data**, or objects, rather than functions and logic.





Key Concepts in OOP

- **Object**:
  - A real-world entity in a program, encapsulating data and behavior.
- **Class**:
  - A blueprint or template for creating objects.
- **Fields (Attributes)**:
  - Data stored in an object.
- **Methods (Functions)**:
  - Behaviors or actions an object can perform.


Attributes
Name
Age
Gender
Occupation
Person
(Class)
Functionality
Walk()
Eat()
Sleep()
Work()

Class: Defines the structure and behavior.

Object: An instance of a class with its own data.

CLASS	OBJECT
Class is a data type	Object is an instance of class.
It generates objects	It gives life to CLASS
Does not occupy memory location	It occupies memory location.
It cannot be manipulated because it is not available in memory	It can be manipulated.

An Object-Oriented Class

- If we think of a real-world object, such as a TV, it will have several features and properties:
  - We do not have to open the case to use it.
  - We have some controls to use it (buttons on the box, or a remote control).
  - It is complete when we purchase it, with any external requirements well documented.
  - The TV will not crash!

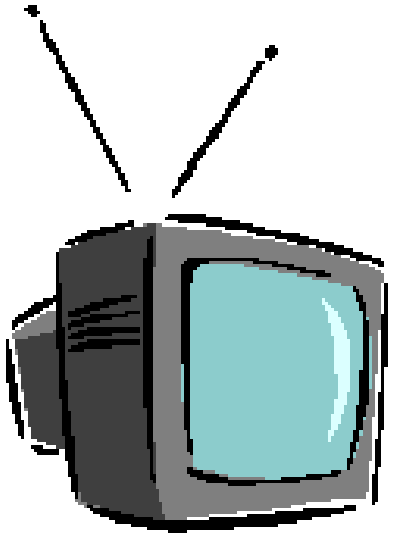
An Object-Oriented Class

- A class should:
  - Provide a well-defined interface
    - such as the remote control of the television.
  - Represent a clear concept - such as the concept of a television.
  - Be complete and well-documented - the television should have a plug and should have a manual that documents all features.
  - The code should be robust - it should not crash, like the TV.

An Object-Oriented Class

- Classes allow us a way to represent complex structures within a programming language. They have two components:
  - ***States, attributes, fields*** - (or data) are the values that the object has.
  - ***Methods*** - (or behaviour) are the ways in which the object can interact with its data, the actions.

Unified Modelling Language (UML)



Example States

Example Methods

«java class»

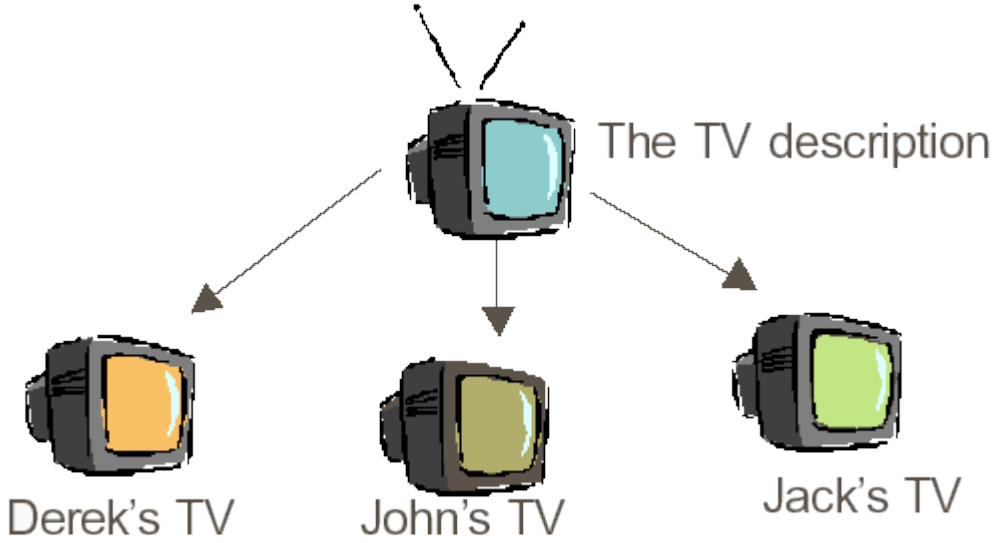
Television

- channelNumber : int
- onOff : boolean
- volumeLevel : int

+ changeChannel (int channel) : void
+ changeVolume (int level) : void
+ muteVolume (boolean isMute) : void
+ powerOff () : void
+ powerOn () : void

An Object

- An ***object*** is an instance of a class.
- class is the description of a concept
- object is the realisation of this description to create an independent distinguishable entity



Practical Applications

OOP is widely used across industries due to its ability to model real-world problems effectively:

- Game Development
  - characters, weapons, and environments are often modeled as objects with attributes and behaviors
  - A character object may have properties like health, speed, and position, and methods like move(), attack(), and heal().

Create Character

- Name
- Gender
- Age
- Class
- Burial Gift
- Face Presets
- Build
- Appearance

Finalize creation

A former herald who journeyed to finish a quest undertaken. Wields a sturdy spear and employs a gentle restorative miracle.

- Knight
- Mercenary
- Warrior
- Herald
- Thief
- Assassin
- Sorcerer
- Pyromancer
- Cleric
- Deprived

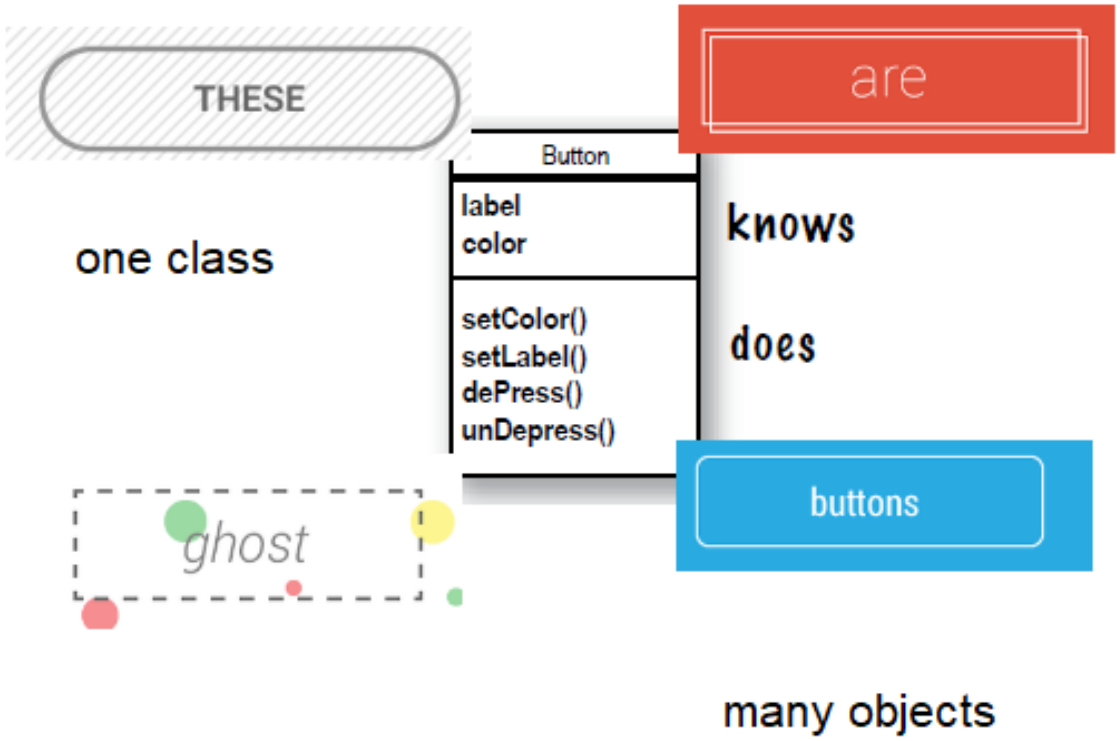
Level	9
Vigor	12
Attunement	10
Endurance	9
Vitality	12
Strength	12
Dexterity	11
Intelligence	8
Faith	13
Luck	11

Select class

A :OK B :Back Y :Switch view R :Rotate LT RT :Zoom R :Reset Camera :Help

Practical Applications

- Mobile Applications
  - Buttons, text fields, and other UI elements are objects, each with attributes (e.g., size, color) and behaviors (e.g., onClick()).
  - **Example:** In Android development, classes like Button or TextView define the behavior and appearance of UI elements.

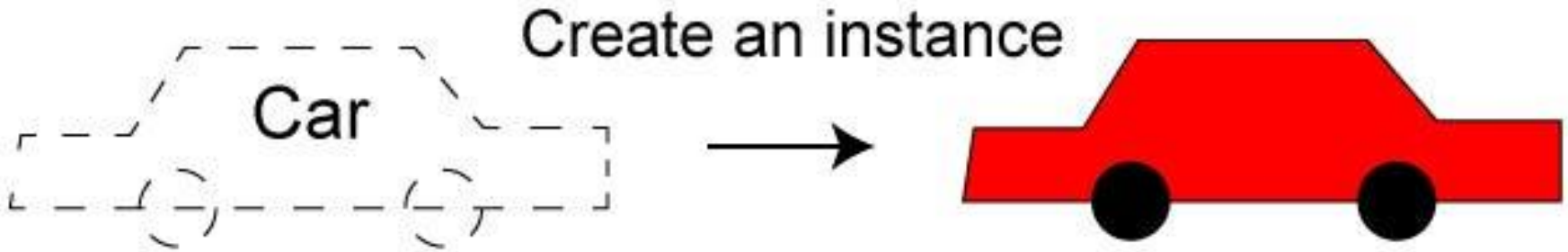


Practical Applications

- Enterprise Systems
  - Enterprise systems like Customer Relationship Management (CRM) or Enterprise Resource Planning (ERP) use OOP to model entities such as customers, orders, and inventory.
  - **Example:** A Customer class might have fields like name and email, and methods like placeOrder() or updateDetails().

Class

Object



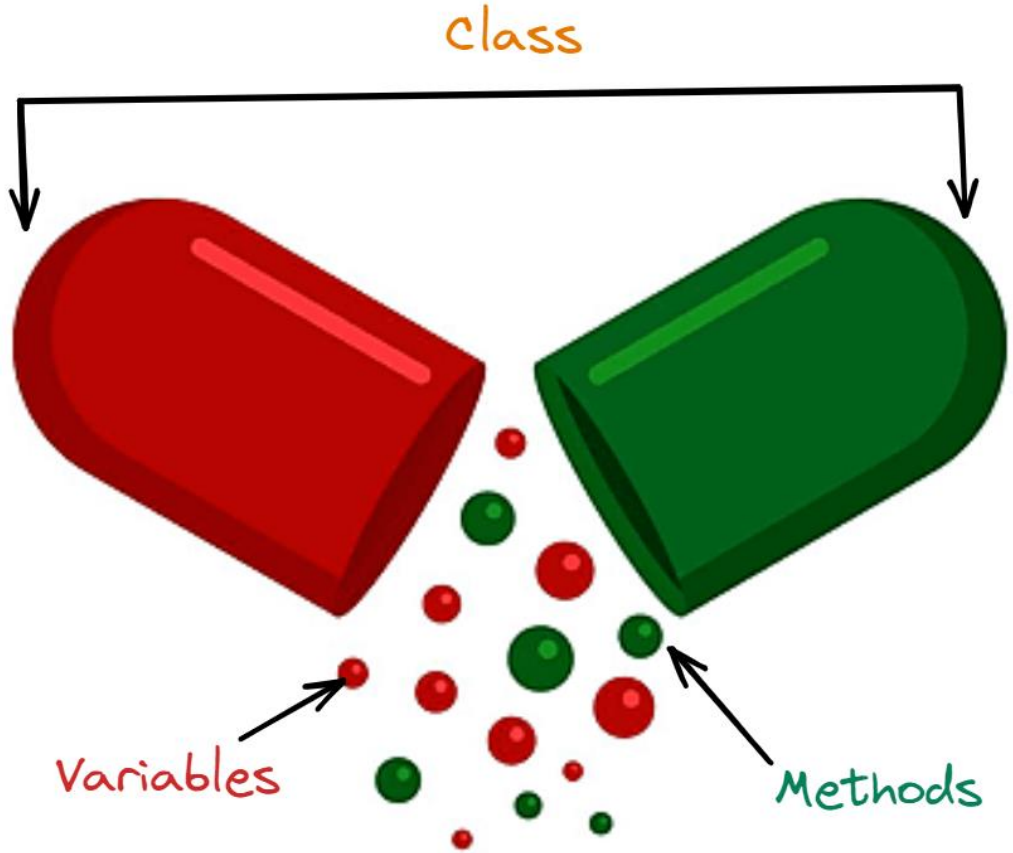
Properties	**Methods** - behaviors
color	start()
price	backward()
km	forward()
model	stop()

Property values	**Methods**
color: red	start()
price: 23,000	backward()
km: 1,200	forward()
model: Audi	stop()

Principles of OOP

Encapsulation

IN - CAPSULE - ation



Encapsulation

- is the concept of hiding the internal details of an object and exposing only the necessary parts through a defined interface
- Protects the internal mechanics of the object
- Allows users to interact with the object without knowing its implementation details.

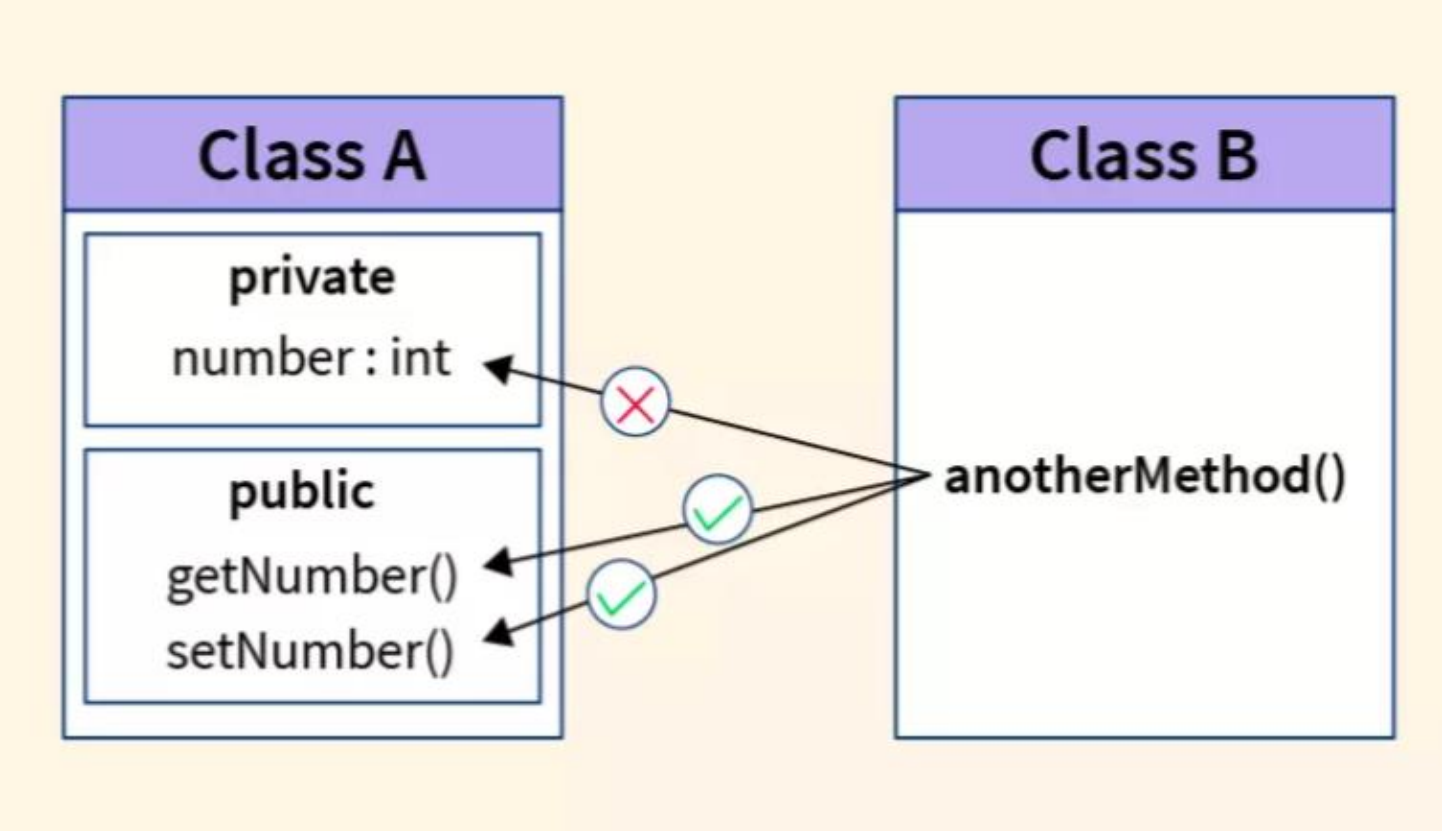
Encapsulation why?

- **Benefits:**
  - **Security**:
    - Prevents unauthorized access to sensitive data
  - **Simplicity**:
    - Users interact with a clean, simplified interface
  - **Code Maintainability**:
    - Encapsulation ensures one part of the code can change without affecting others.

- **Example:** Python's range()
    - You use *for i in range range(1, 10)* without needing to know the internal C implementation.
    - Source: CPython Implementation.

Getters and Setters

- **Getting**:
  - Retrieve information using methods
- **Setting**:
  - Update information using methods.



Encapsulation in a Bank Account

- **States (Attributes)**
  - name, id, balance

- **Methods (Behavior)**
  - **Check Balance**:
    - Provides balance without direct access to the raw value.
  - **Deposit Money**:
    - Adds to balance while ensuring correctness.
  - **Withdraw Money**:
    - Prevents overdrawing or invalid operations.

Why Encapsulation Matters

- **Protection from Errors**
  - Prevents external code from accidentally or maliciously altering object data.
- **Rules of Interaction**
  - Objects interact through defined interfaces, ensuring predictable behavior.
- **Scalability**
  - In large projects, encapsulation prevents one developer’s changes from breaking another’s code.

Summary of Encapsulation

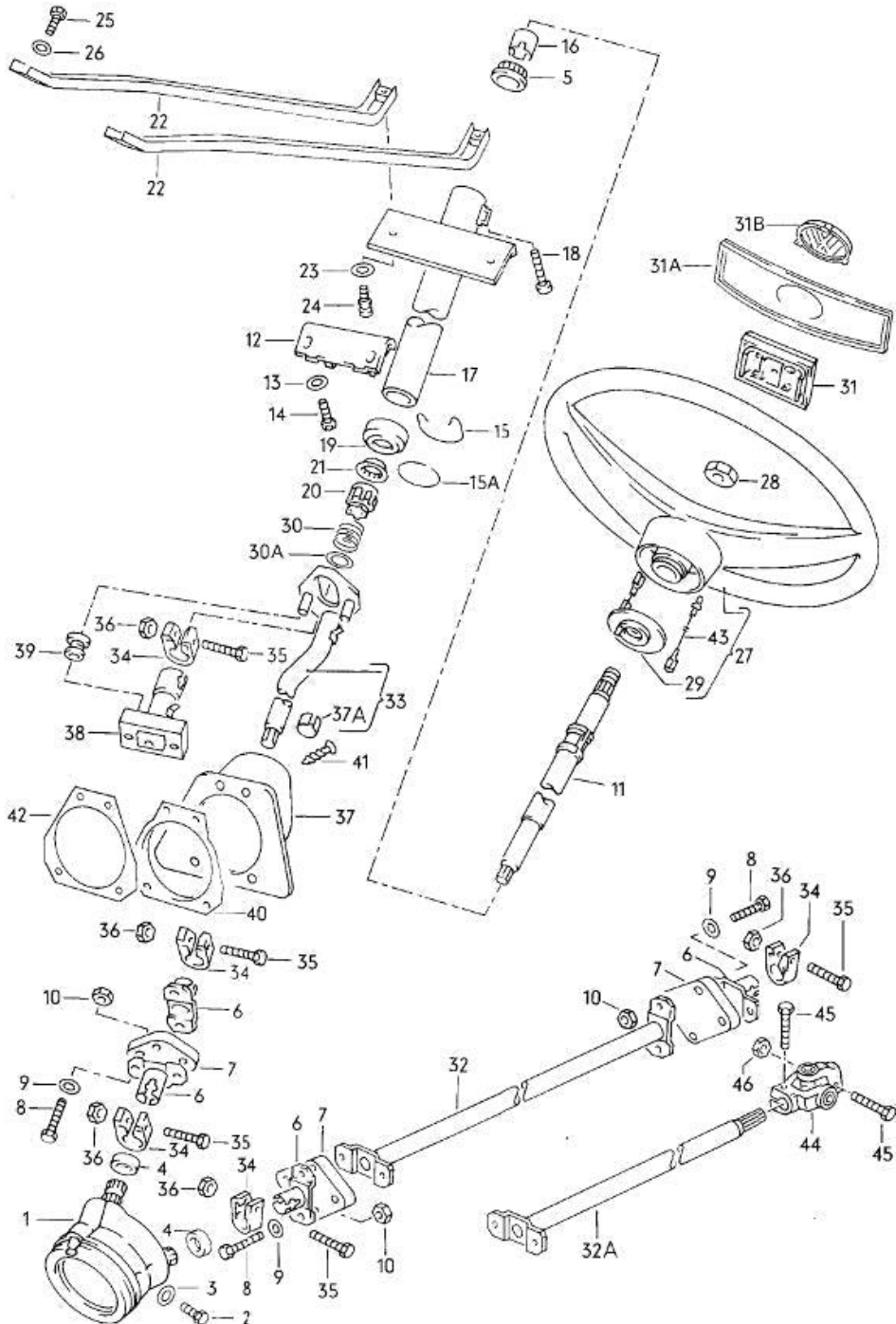
- **Encapsulation Achieves:**
  - Hiding data implementation details.
  - Providing controlled access through methods.
  - Enhancing security and maintainability.
- **Practical Benefits:**
  - Keeps the programmer in control of data.
  - Prevents programs from ending up in unwanted states.

Abstraction

What is Abstraction?

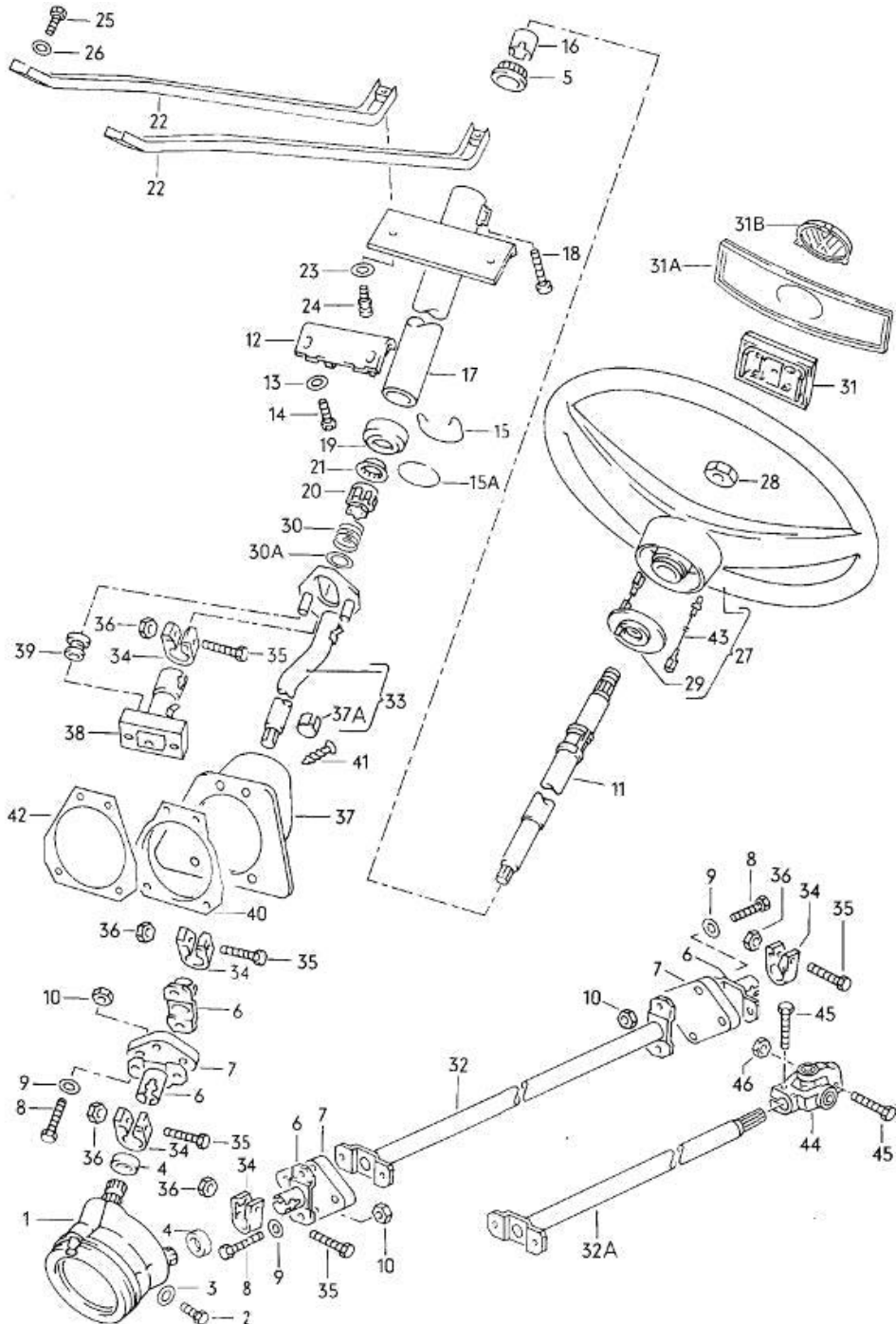
- Abstraction is the process of showing only essential details while hiding the internal implementation.
- Users interact with the system based on what they need to know, not how it works internally.

- **Real-World Analogy: A Car**
  - **Steering Wheel**:
    - You don’t need to know the mechanics of turning the wheels, only that turning the wheel changes direction.
  - **Gas/Brake Pedals**:
    - You press them without worrying about how they control the engine or brakes.
  - **Fuel Gauge**:
    - Tells you how much petrol is left; no need to understand how the sensor works.



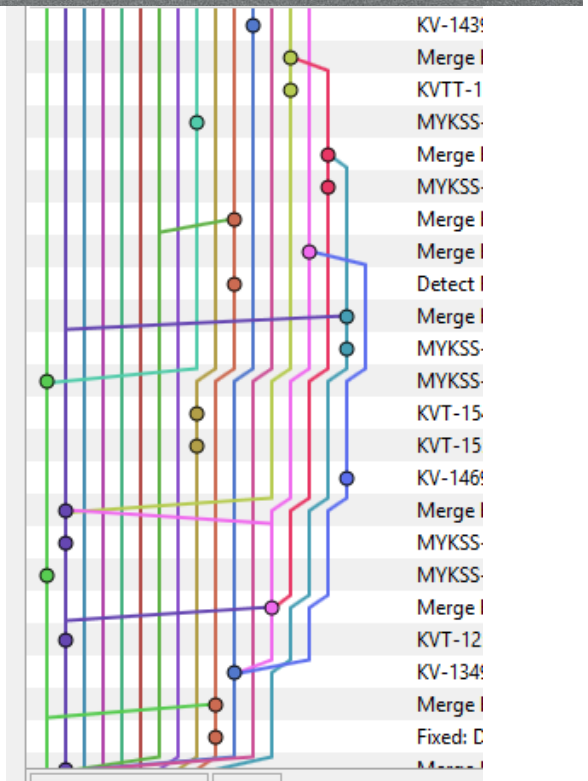
Who Needs to Know What?

- **Engineers:**
  - Design and implement the internal systems (e.g., the forces and mechanics behind the steering wheel).
- **Mechanics:**
  - Maintain and repair the system, understanding its key components.
- **Drivers (Users):**
  - Use the system with no concern for the underlying complexity, only its functionality.
- **In OOP Terms:**
  - The **engineers** are the developers of the class.
  - The **users** interact with objects and methods through an interface.



Abstraction

- **Scenario**: Building a Self-Driving Car
  - Imagine a team of 100 programmers working together.
  - Each programmer works on a specific module or object.
- **Your Task:**
  - Write the safety system for the car.
    - Check if the seatbelt is engaged (using objects written by another programmer).
    - Trigger airbags during a crash.
    - Monitor sensors for collision detection.
- **Interaction Example:**
  - Your module uses **interfaces** to interact with other objects, such as the seatbelt or airbag systems, without needing to know how they're implemented.
- **Outcome:**
  - Simpler collaboration.
  - Each programmer focuses only on their module while <u>trusting</u> the interfaces of other components.



What is an Interface?

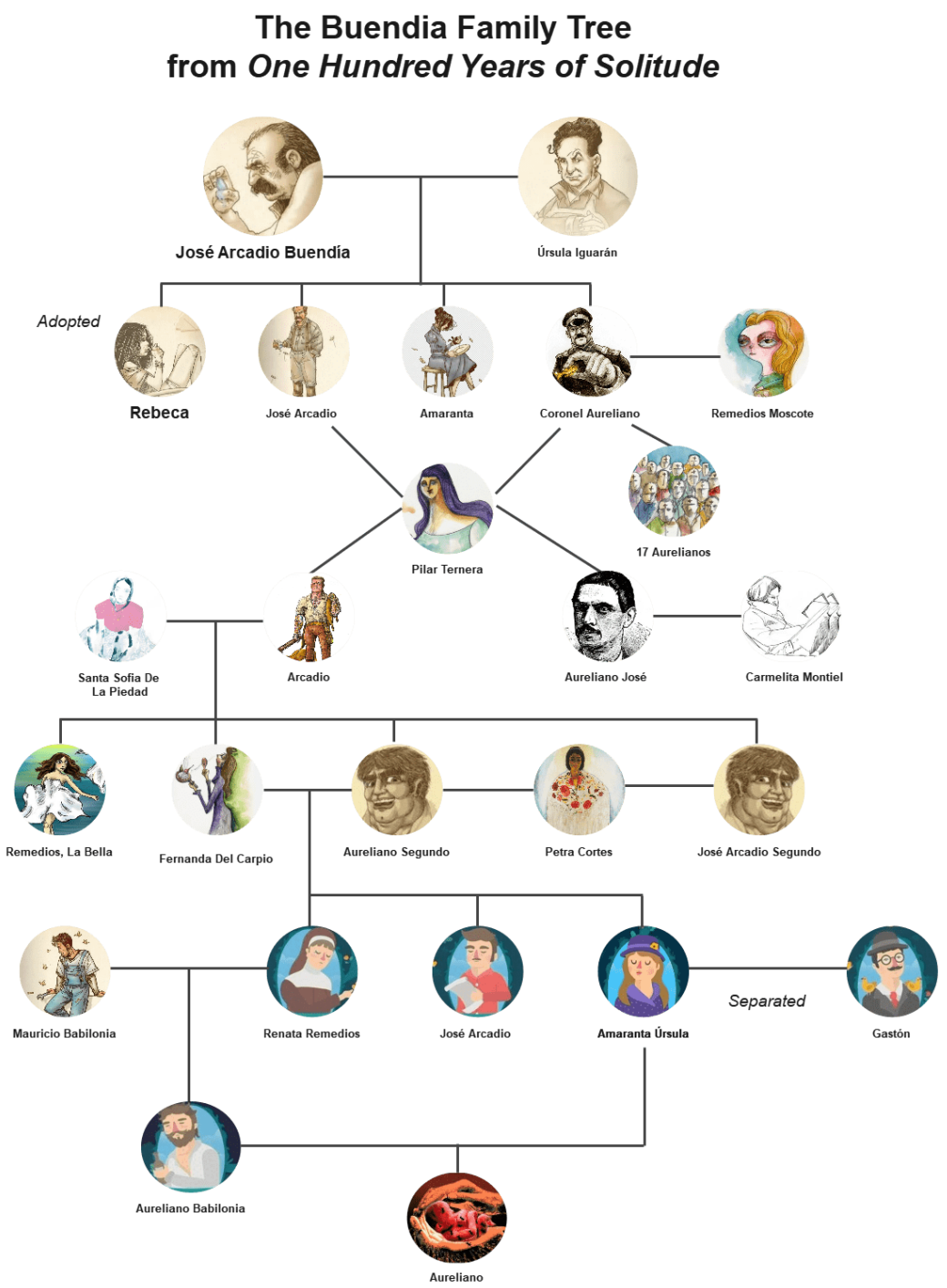
- An interface is the way sections of code or objects communicate with each other.
- It defines *what* actions are possible, not *how* they are performed.

- **Key Benefits**
  - **Modularity**: Each section of code is isolated and interacts only through the interface.
  - **Reusability**: Components with clear interfaces can be reused in other projects.

Abstraction summary

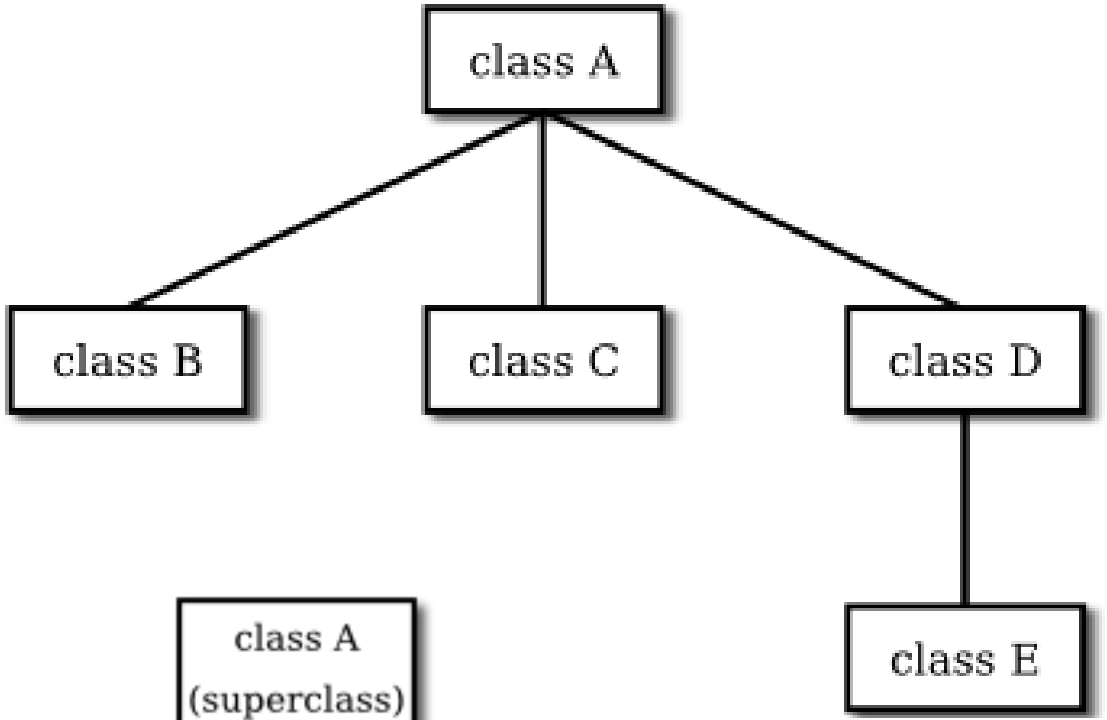
- **Abstraction:**
  - Hides the complexity of objects.
  - Focuses on essential details.
- **Interfaces:**
  - Define how objects communicate.
  - Simplify collaboration and reusability.

Inheritance

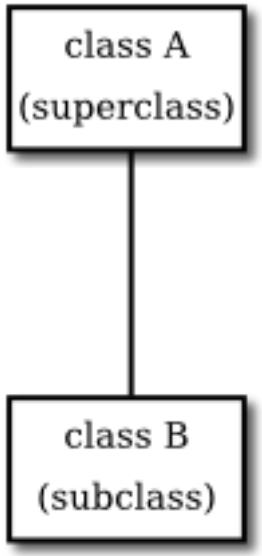


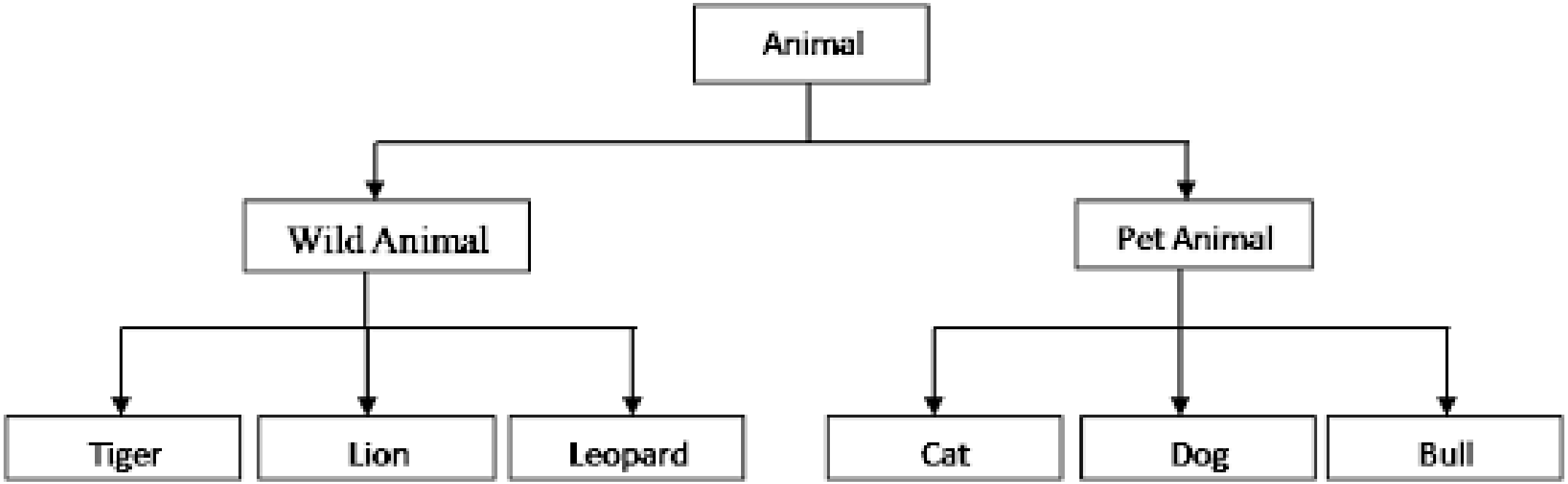
What is Inheritance?

- Inheritance allows a class (child class) to acquire properties and behaviors (methods) from another class (parent class).

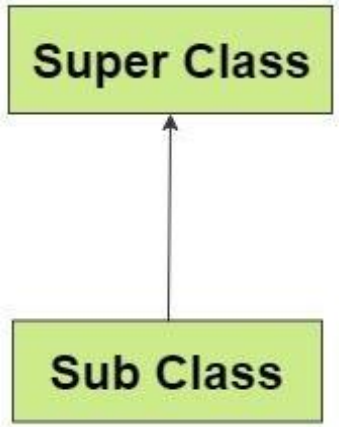


- **Key Concepts**
  - **Parent Class (Superclass):**
    - The class whose properties and methods are inherited.
  - **Child Class (Subclass):**
    - The class that inherits from the parent class.

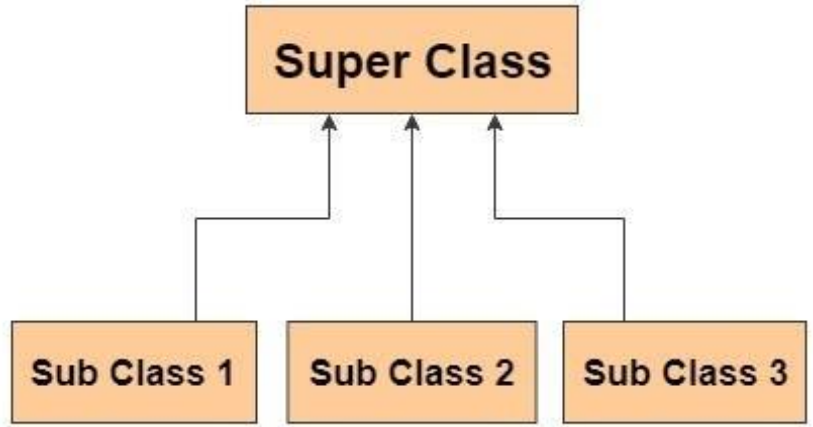



Animal
Wild Animal
Pet Animal
Tiger
Lion
Leopard
Cat
Dog
Bull

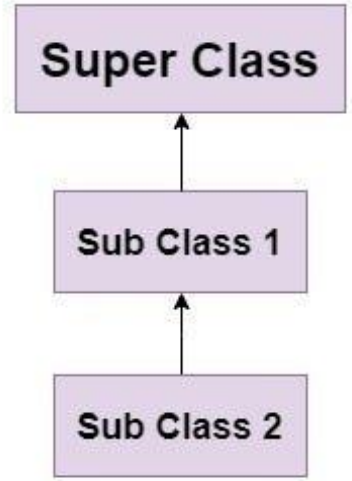
Single Inheritance



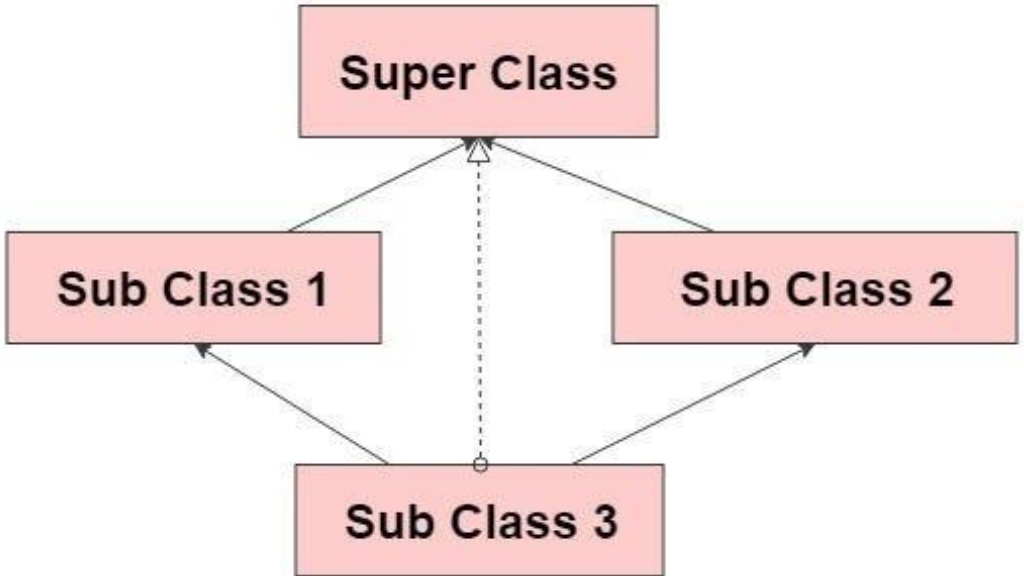
Hierarchial Inheritance



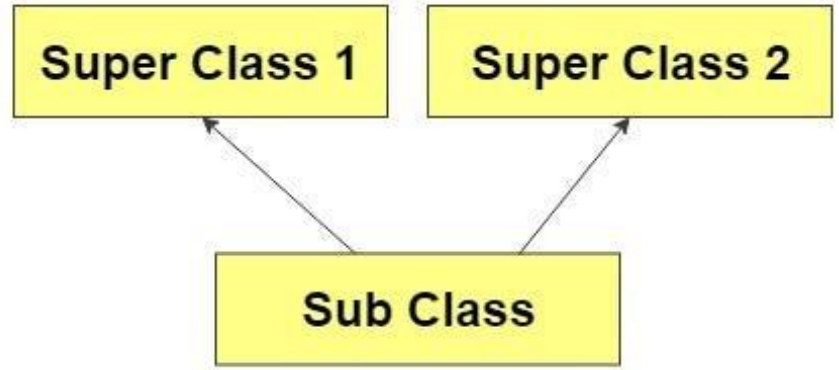
MultiLevel Inheritance

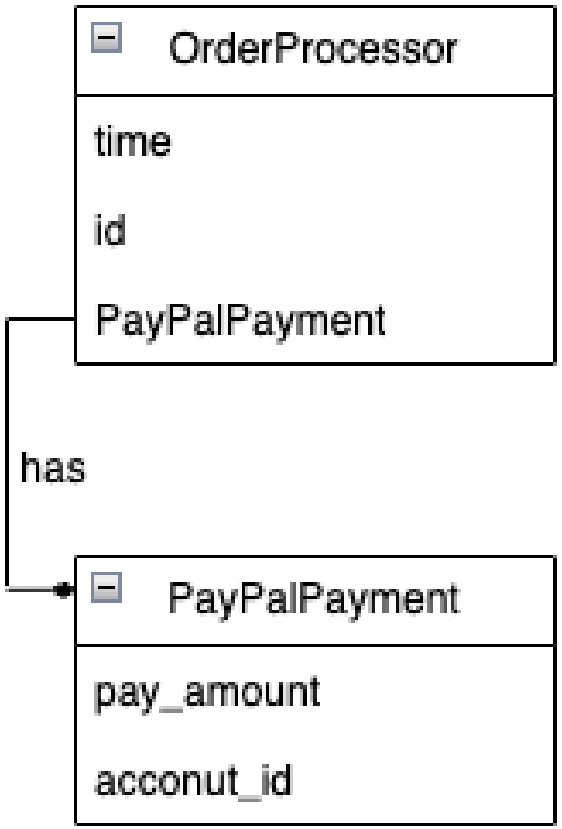


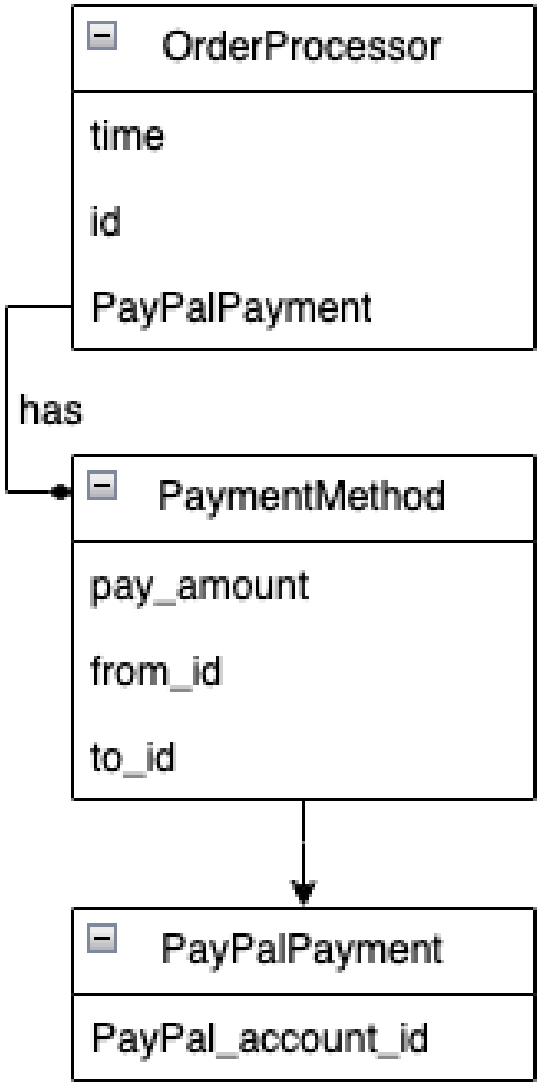
Hybrid Inheritance

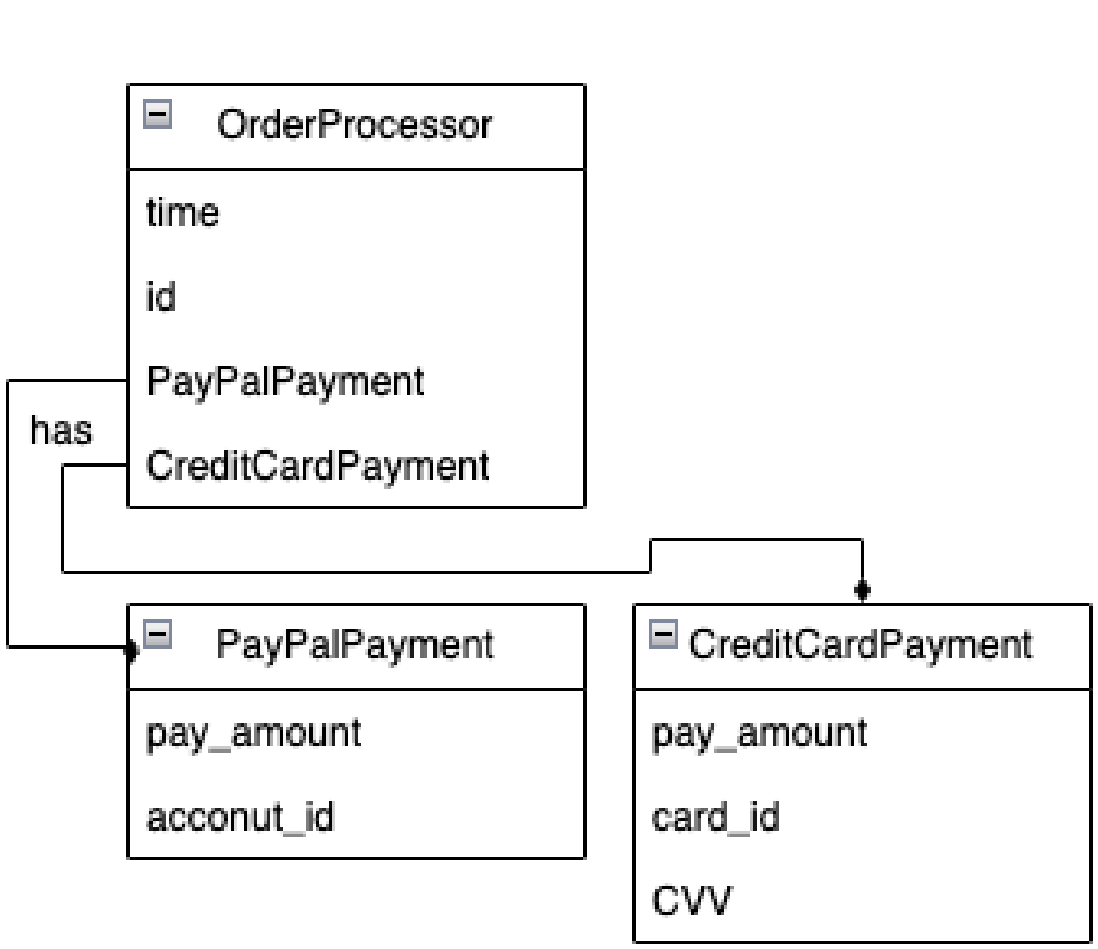


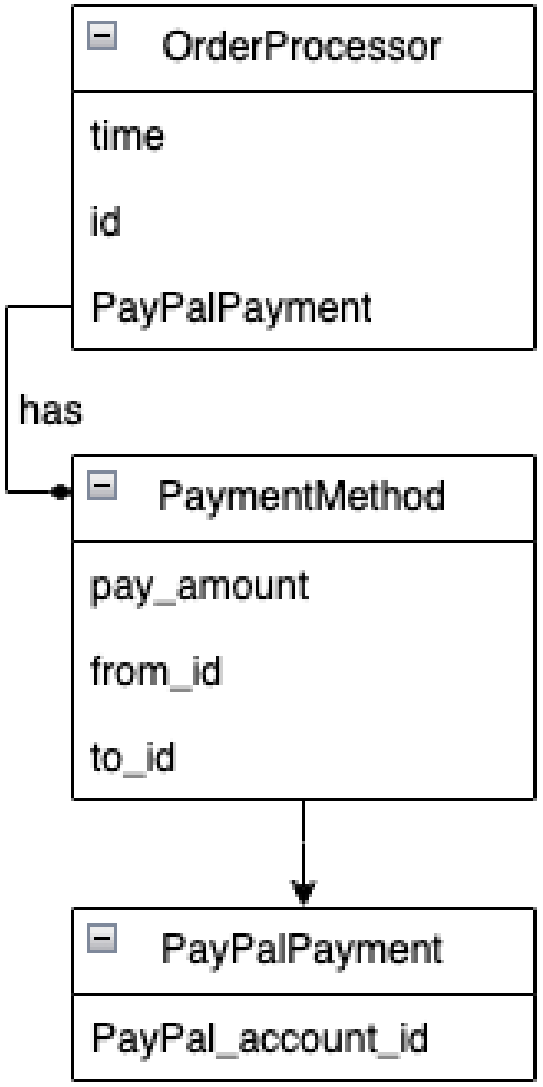
Multiple Inhertance

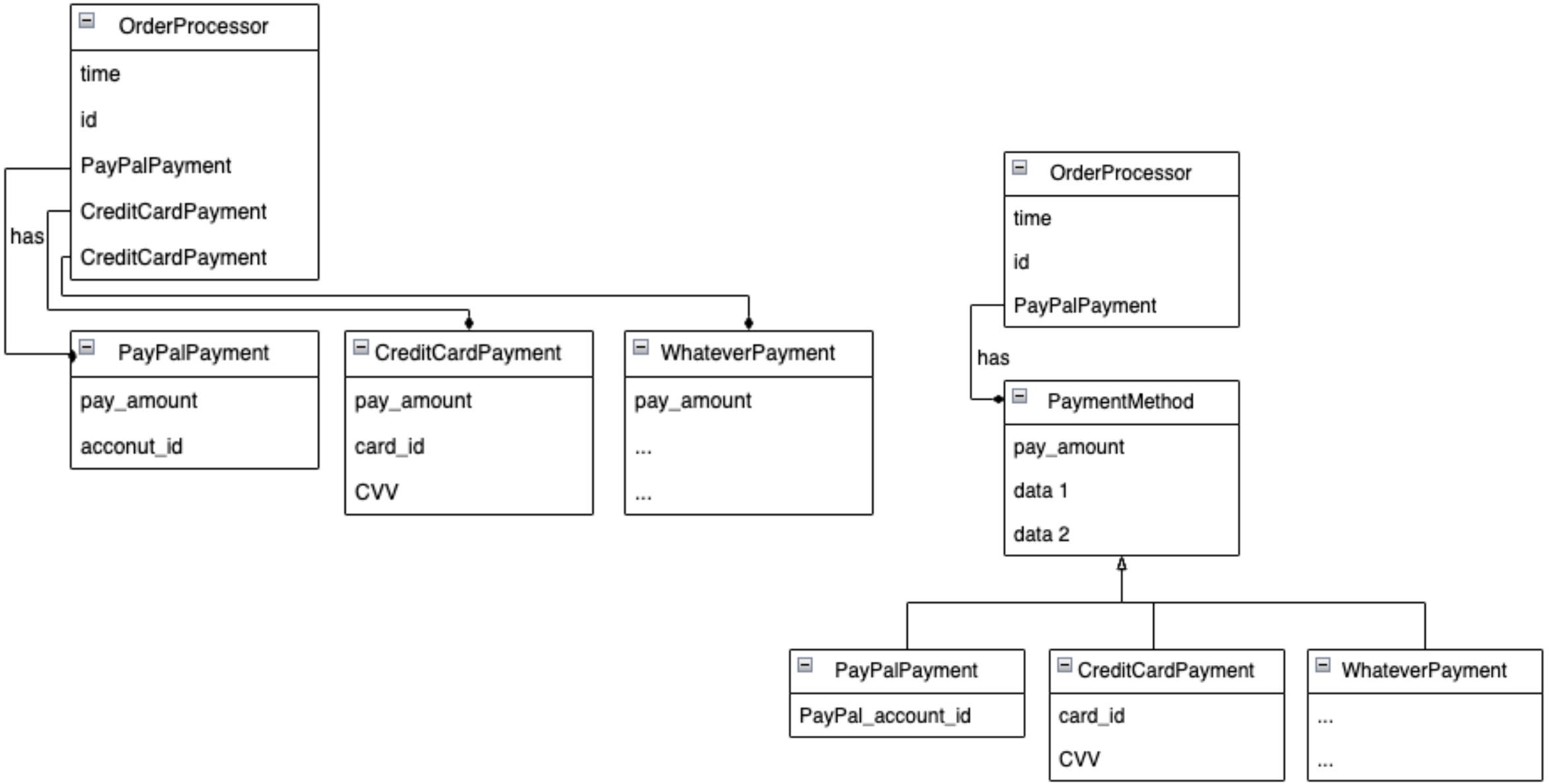



OrderProcessor
time
id
PayPalPayment
has
PayPalPayment
pay_amount
acconut_id


OrderProcessor
time
id
PayPalPayment
has
PaymentMethod
pay_amount
from_id
to_id
PayPalPayment
PayPal_account_id


OrderProcessor
time
id
PayPalPayment
CreditCardPayment
has
PayPalPayment
pay_amount
acconut_id
CreditCardPayment
pay_amount
card_id
CVV


OrderProcessor
time
id
PayPalPayment
has
PaymentMethod
pay_amount
from_id
to_id
PayPalPayment
PayPal_account_id


OrderProcessor
time
id
PayPalPayment
CreditCardPayment
CreditCardPayment
has
PayPalPayment
pay_amount
acconut_id
CreditCardPayment
pay_amount
card_id
CVV
WhateverPayment
pay_amount
...
...
OrderProcessor
time
id
PayPalPayment
has
PaymentMethod
pay_amount
data 1
data 2
PayPalPayment
PayPal_account_id
CreditCardPayment
card_id
CVV
WhateverPayment
...
...

Why Use Inheritance?

1.Reusability

- Code written for a parent class can be reused in child classes, reducing duplication.

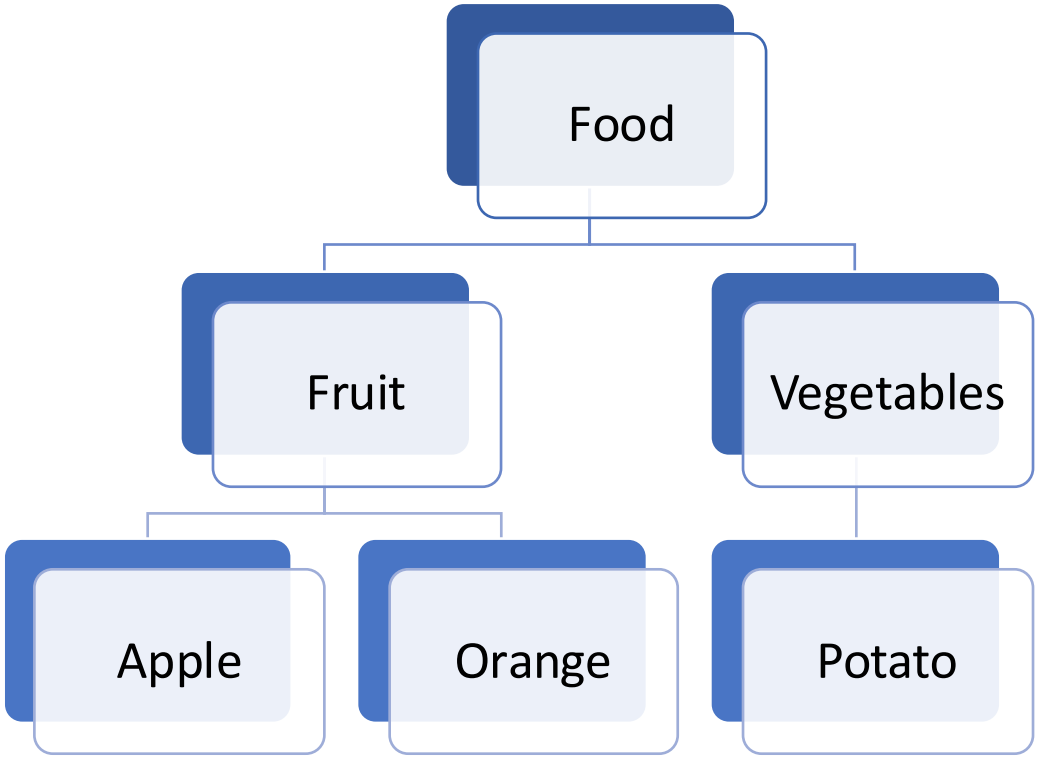
2.Extensibility

- Child classes can add or override properties and methods to extend functionality.

3.Maintainability

- Centralized code in the parent class makes updates easier and avoids redundancy.

Access Modifiers

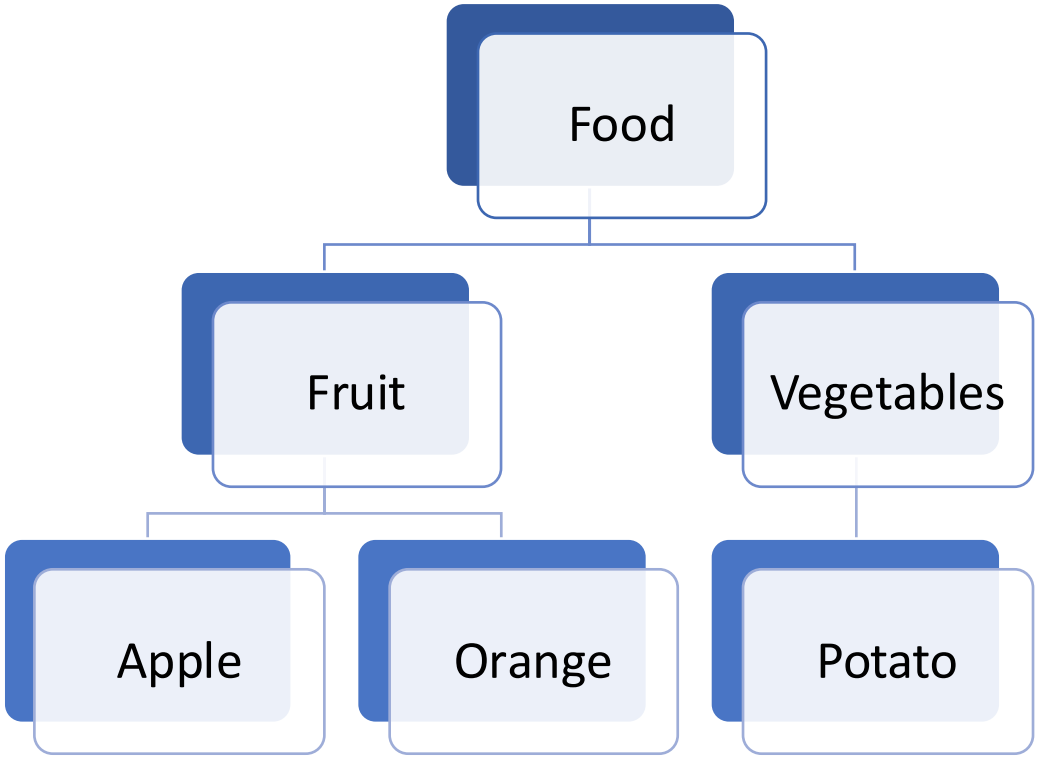


- Access modifiers determine how attributes and methods of a class can be accessed or **inherited**.
- They define the visibility and scope of class members.
- Control how child classes and other parts of the program interact with a class.

Types of Access Modifiers

- **Public:**
  - Accessible from any class in the hierarchy or external code.
- **Protected:**
  - Accessible within the parent and child classes.
- **Private:**
  - Accessible only within the defining class.

Public

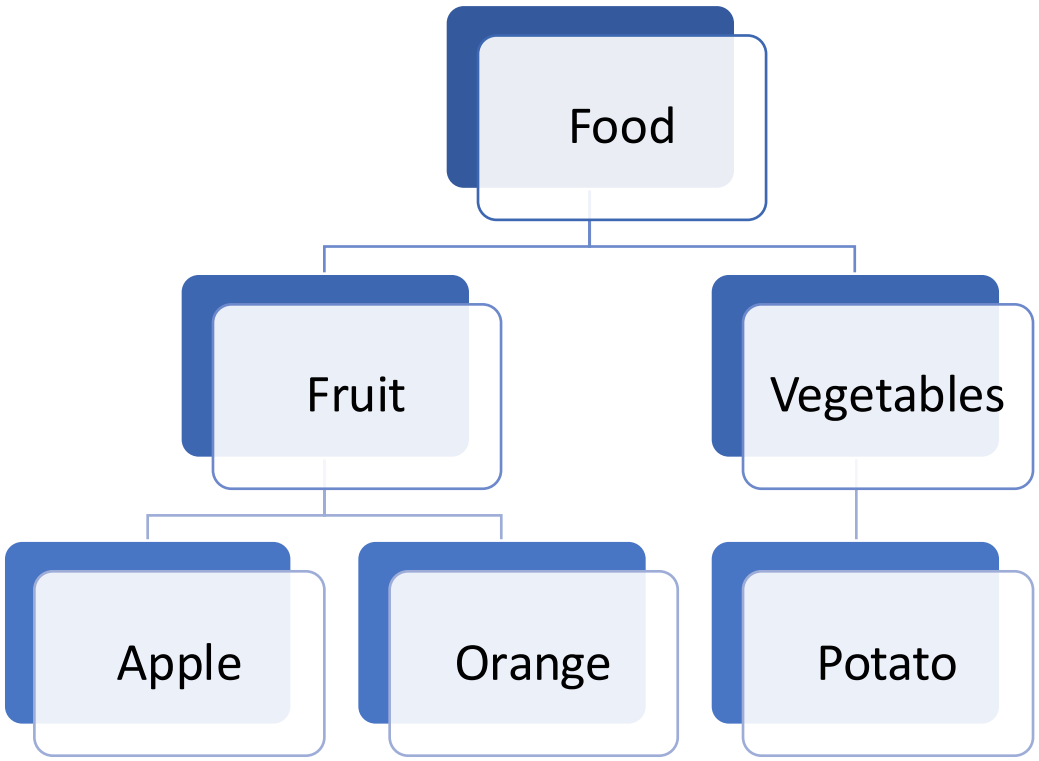


Public attributes can be accessed from anywhere:

- Child classes.
- External code.
- *Parent class (be careful!)*

Public attributes make sharing information between parent and child classes seamless, but they can expose sensitive data to unintended modifications.

Protected

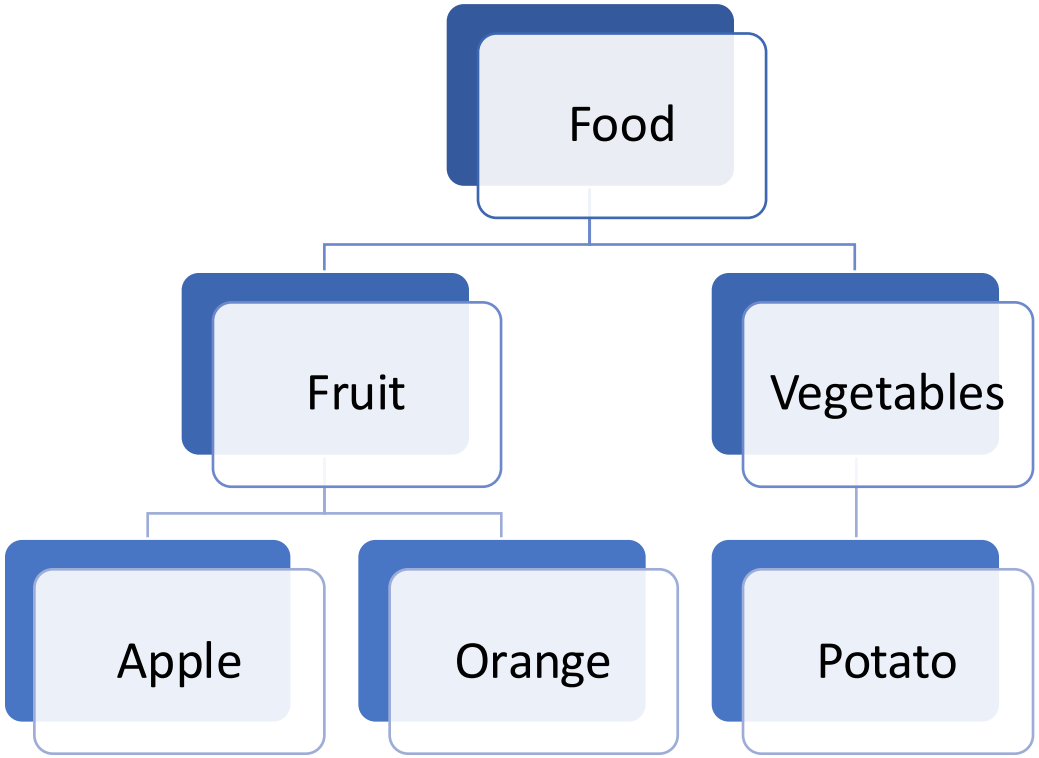


Protected attributes are accessible only within the parent class and its child classes.

They are not accessible to external code or instances of the classes.

Protected attributes ensure data is shared only within the hierarchy, making the structure modular and secure.

Private



Private attributes are accessible only within the parent class itself.

They are not inherited by child classes or accessible from external code.

Private attributes are used to hide sensitive information, even from child classes. They protect critical data and ensure encapsulation.

Summary of Access Control

- Use public attributes for general information sharing.
- Use protected attributes for controlled sharing within the hierarchy.
- Use private attributes to encapsulate sensitive data.

Modifier	Class	Package	Subclass	World
public	Y	Y	Y	Y

Summary of Access Control

- Use public attributes for general information sharing.
- Use protected attributes for controlled sharing within the hierarchy.
- Use private attributes to encapsulate sensitive data.

Modifier	Class	Package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N

Summary of Access Control

- Use public attributes for general information sharing.
- Use protected attributes for controlled sharing within the hierarchy.
- Use private attributes to encapsulate sensitive data.

Modifier	Class	Package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N
	Y	Y	N	N

Summary of Access Control

- Use public attributes for general information sharing.
- Use protected attributes for controlled sharing within the hierarchy.
- Use private attributes to encapsulate sensitive data.

Modifier	Class	Package	Subclass	World
public	Y	Y	Y	Y
protected	Y	Y	Y	N
	Y	Y	N	N
private	Y	N	N	N

Polymorphism

What is Polymorphism?

- Polymorphism allows methods to take many forms.
- A single function name or method can behave differently based on the context.

- Enhances code flexibility and reusability.

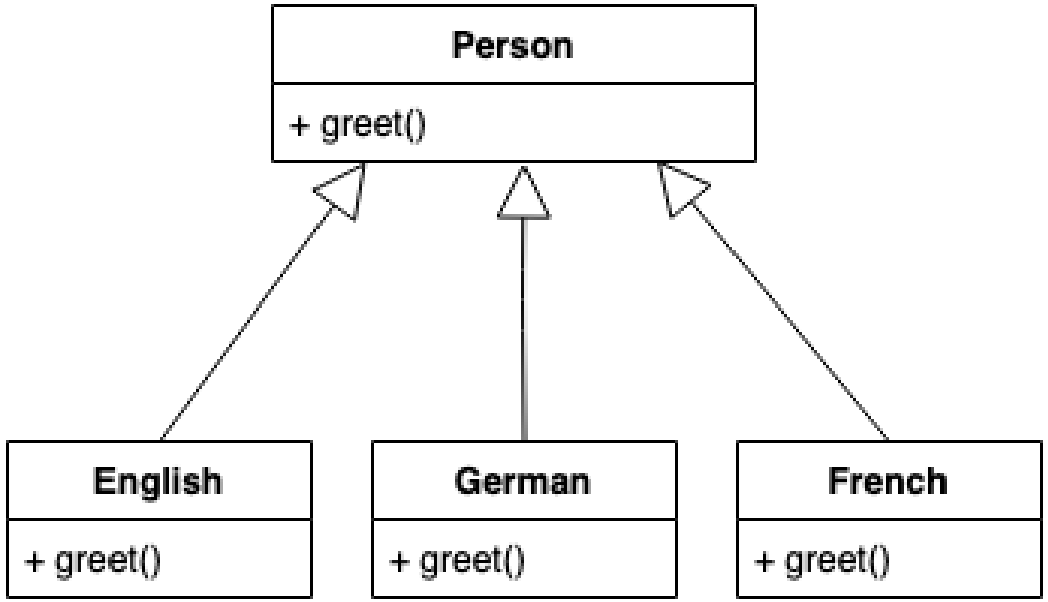
Types of Polymorphism

- **Dynamic Polymorphism**
  - Occurs **during runtime**.
  - Subclasses **override** a method from the parent class.
  - **Key Feature:**
    - Same method name, different implementation in the subclass.

- **Static Polymorphism**
  - Occurs **during compile time**.
  - Achieved through **method overloading**.
  - **Key Feature:** Same method name but different:
    - Number of parameters.
    - Types of parameters.
    - Order of parameters.

Dynamic Polymorphism

- A parent class Shape defines a greet() method.
- Subclasses **override** the method to provide their specific implementation.
- The greet() method behaves differently based on the object type (e.g., English or German).
- The implementation is resolved at runtime.



Static Polymorphism - Method **Overloading**

limited in python

- A class defines multiple methods **with the same name** but different parameter lists.
- Example: Adding vectors
  - Add two 2D vectors.
  - Add two 3D vectors.
  - Add a list of vectors.

python

```python
max(a, b)                    # Compares two numbers
max([1, 23, 12, 4, 1])       # Finds the max in a list
max(a, b, c, d)              # Compares multiple numbers
```

Comparing Polymorphism Types

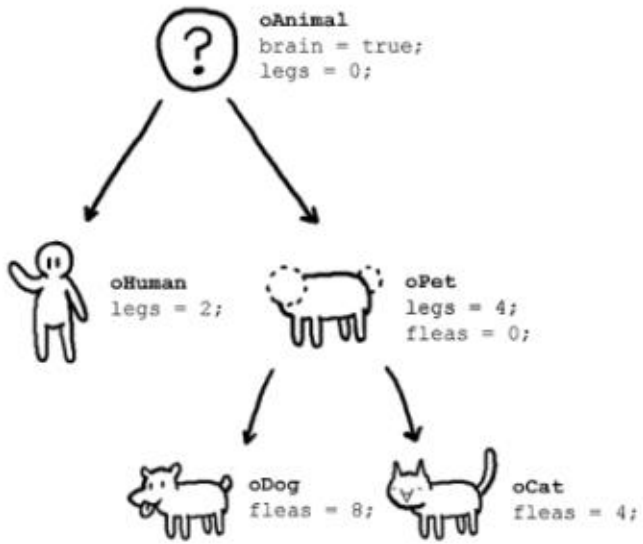
Aspect	Dynamic Polymorphism	Static Polymorphism
Timing	Resolved at runtime	Resolved at compile time
Mechanism	Method **overriding** in subclasses	Method **overloading** in the same class
Flexibility	Behavior depends on object type	Behavior depends on arguments
Use Case	Used in inheritance-based designs	Used for handling different parameter setups

- Enables behavior customization in subclasses.
- Makes methods more versatile with flexible parameterization.
- Polymorphism allows us to design systems that are extensible, reusable, and easy to maintain.

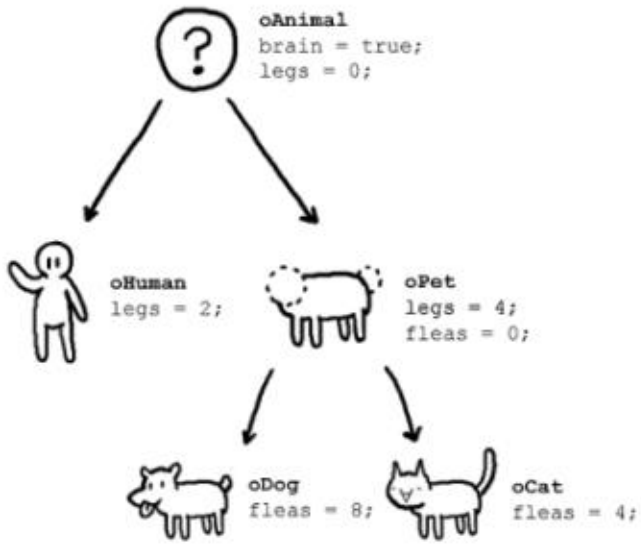
4 Concepts of OOP

Encapsulation Abstraction Inheritance Polymorphism

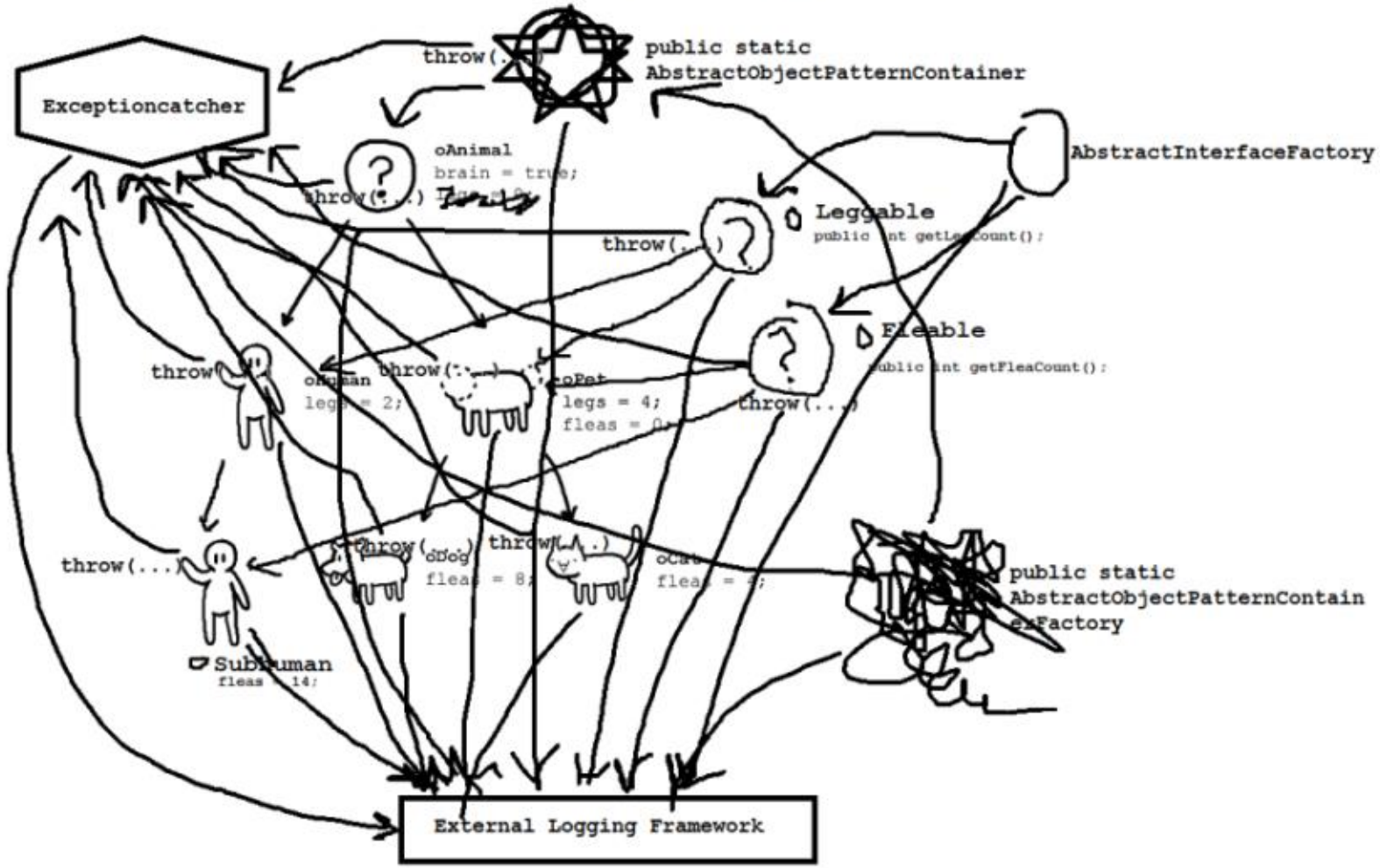
What OOP users claim



What OOP users claim



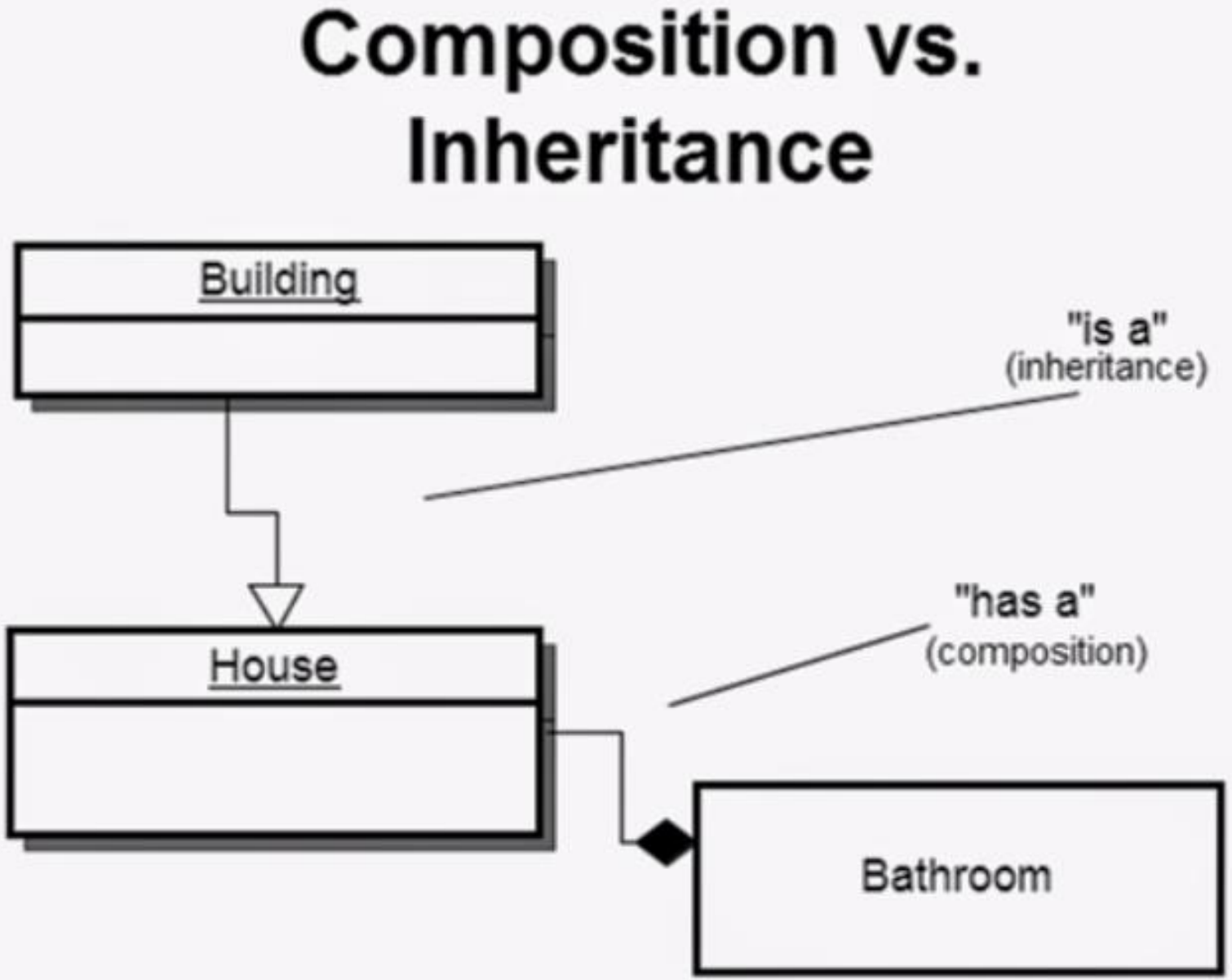
What actually happens



Common Pitfalls in OOP

Overuse of Inheritance

- **Common Mistake:**
  - Using inheritance for code reuse without considering the "is-a" relationship.
- **Impact:**
  - Creates tightly coupled classes that are difficult to modify or extend.
  - Leads to complex and fragile hierarchies.
- **Example:**
  - Avoid inheriting a Car class from a Vehicle class if you only need a few shared attributes but don’t intend to use the "is-a" relationship.
- **Best Practice:**
  - Use **composition** over inheritance when the "has-a" relationship makes more sense.

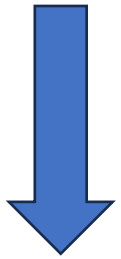


Composition over inheritance

Poor Encapsulation

- **Common Mistake:**
  - Leaving class attributes public, exposing them to unintended modification.
- **Impact:**
  - Reduces control over data integrity.
  - Increases risk of bugs due to unexpected changes.
- **Example:**
  - Directly modifying an object’s attributes instead of using methods.
- **Best Practice:**
  - Use **private** or **protected** attributes with **getters** and **setters**.
  - Ensure sensitive data is only accessible through well-defined interfaces.

```
function transfer(amount, fromAccount, toAccount) {
    fromAccount.balance = fromAccount.balance - amount;
    toAccount.balance = toAccount.balance + amount;
}
```

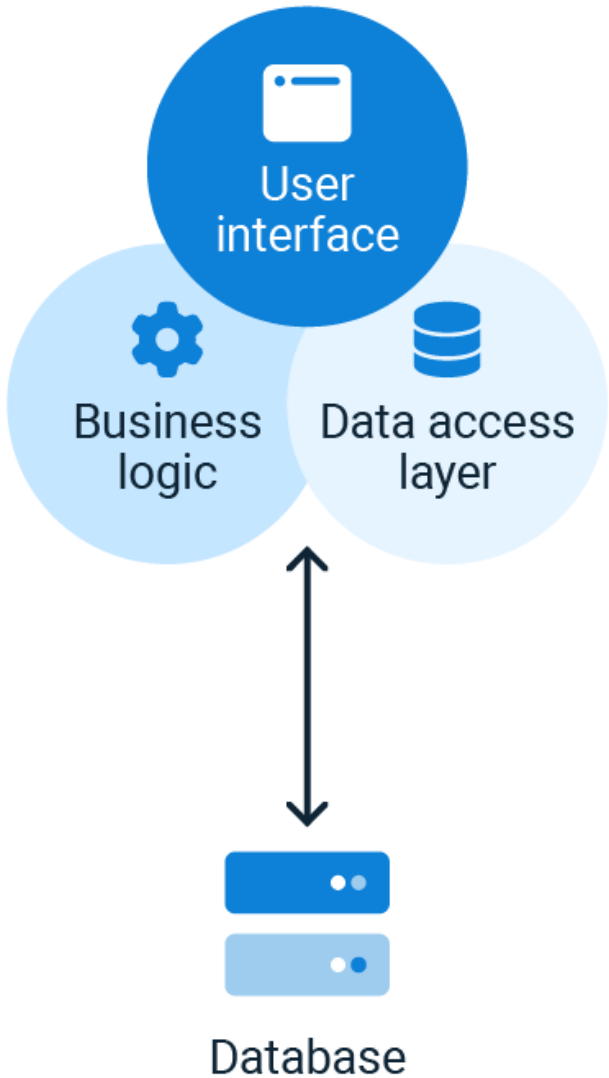


```
function transfer(amount, fromAccount, toAccount) {
    fromAccount.withdraw(amount);
    toAccount.deposit(amount);
}
```

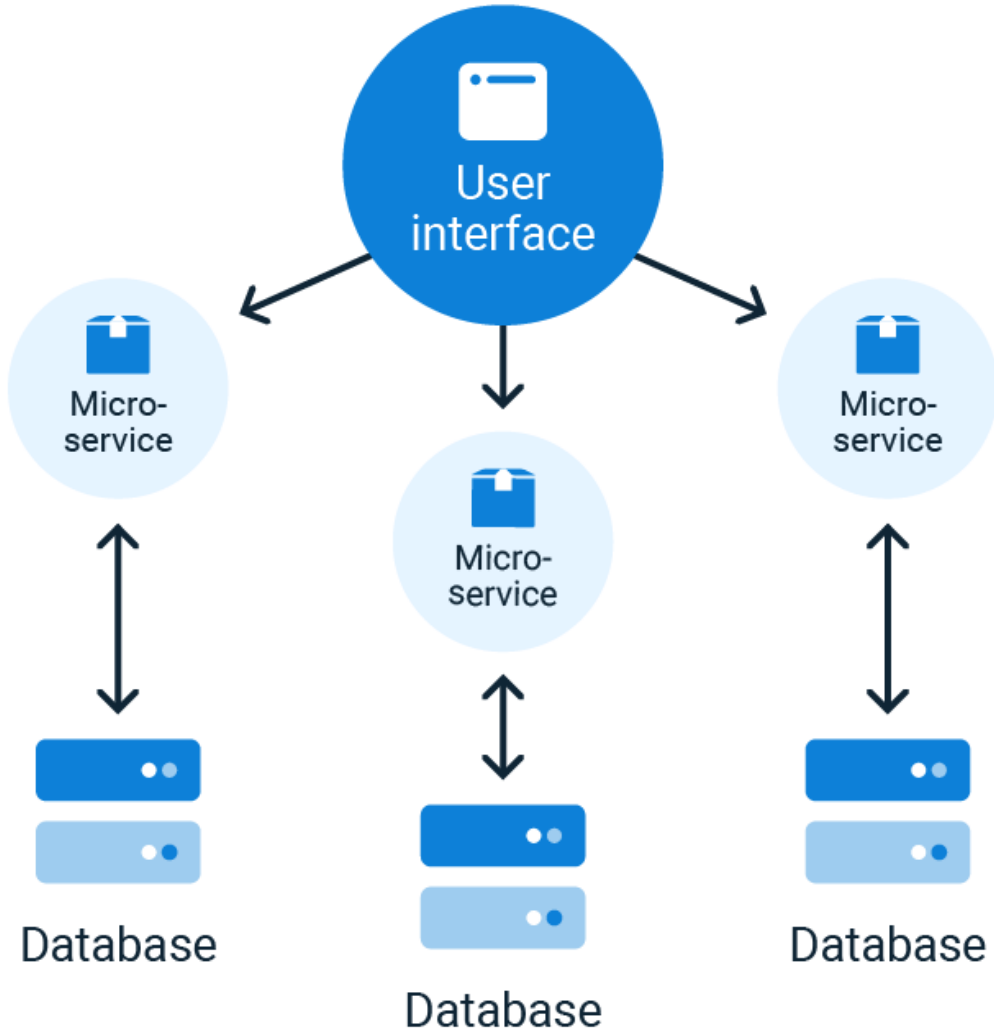
Large, Monolithic Classes

- **Common Mistake:**
  - Packing too much functionality into a single class, often called a "God" class.
- **Impact:**
  - Difficult to understand, maintain, and test.
  - Violates the **Single Responsibility Principle (SRP)**.
- **Best Practice:**
  - Break down large classes into smaller, cohesive classes that each focus on a specific responsibility.

Monolithic Architecture

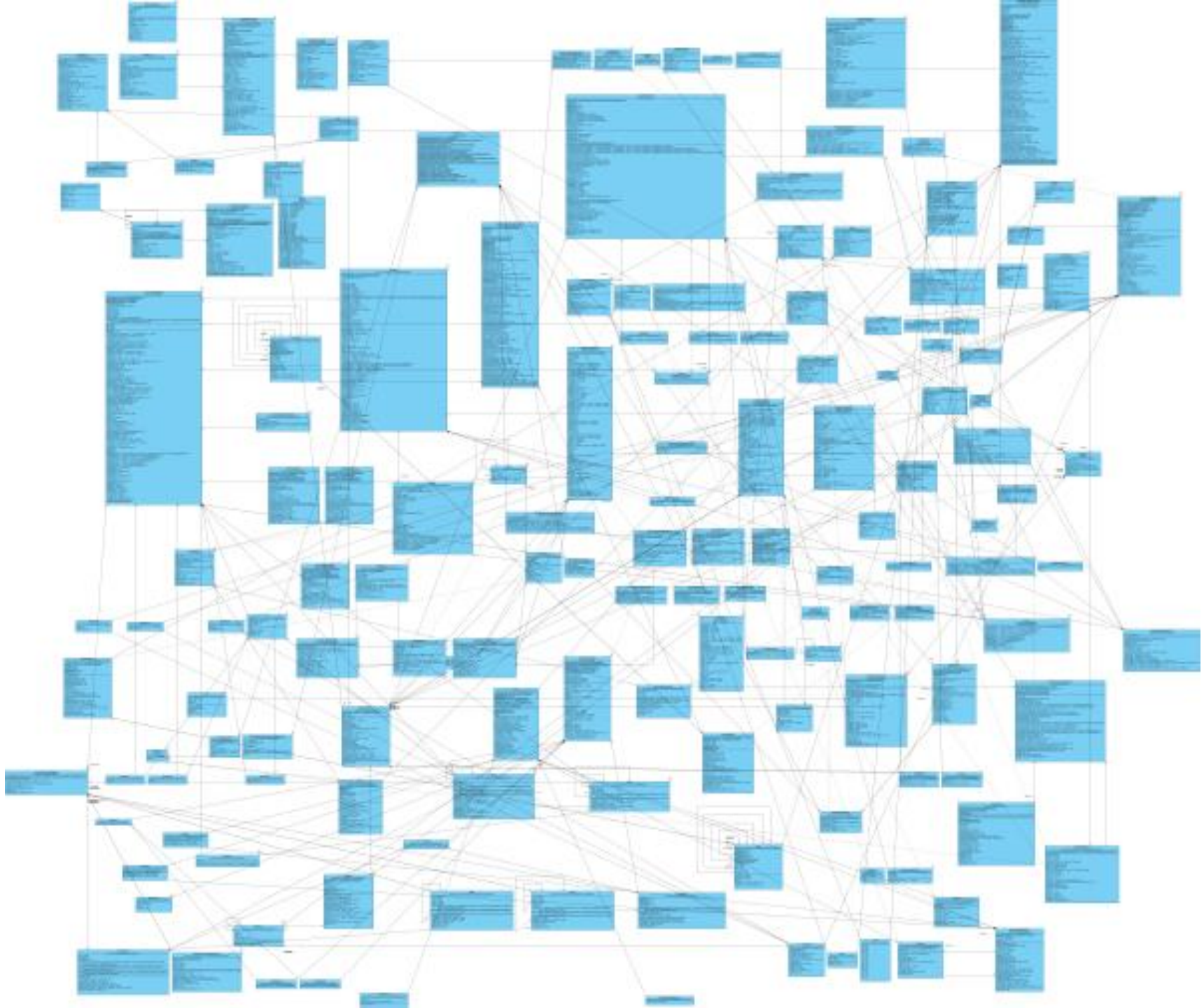


Microservice Architecture



Overcomplicating Design

- **Common Mistake:**
  - Overengineering with excessive use of inheritance, interfaces, or abstract classes.
  - Trying to predict future needs instead of solving the current problem.
- **Impact:**
  - Leads to unnecessary complexity and difficulty in understanding or modifying the code.
- **Best Practice:**
  - Follow the **KISS Principle** (Keep It Simple, Stupid).
  - Design systems to address the current requirements and evolve them when needed.



Ignoring Polymorphism

- **Common Mistake:**
  - Relying on conditional statements (if, switch) instead of polymorphism to handle behavior across subclasses.
- **Impact:**
  - Leads to repetitive and hard-to-maintain code.
  - Fails to leverage the full power of OOP principles.
- **Best Practice:**
  - Use **method overriding** in subclasses to encapsulate behavior variations.
  - Replace conditional logic with polymorphic method calls.

Python and OOP

- While Python is a powerful and flexible object-oriented programming (OOP) language, it does have some limitations compared to other languages like Java, C++, or C#

- Lack of True Encapsulation
  - Python does not enforce strict encapsulation, as there are no true private attributes.
  - Attributes are harder to access but not inaccessible
- No Compile-Time Type Checking
  - Python is dynamically typed, so type errors in method calls or attribute assignments are not caught until runtime

Python and OOP

- No Method Overloading
  - Python does not natively support method overloading
- Performance
  - Python's dynamic nature makes object creation and manipulation slower than in statically typed languages like C++ or Java.
  - Heavy OOP usage in Python may not be ideal for performance-critical applications.
- Multiple Inheritance Complexity

Python and OOP

- Limited Support for Access Modifiers
  - Python only provides limited support for access control compared to languages like C++ (public, protected, private)
  - Access control is achieved mostly through naming conventions.
- No Abstract Class
  - no inherent support for abstract methods
- Large Memory Footprint
  - Python objects are relatively memory-intensive due to the additional metadata required for dynamic typing and flexibility.